



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
Государственное бюджетное профессиональное
образовательное учреждение города Москвы
«Колледж малого бизнеса № 4»
(ГБПОУ КМБ № 4)

Работа допущена к защите

Заместитель директора

_____ / И.Ю. Атрошенко

« ____ » _____ 2026 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

«Разработка и оптимизация высоконагруженного приложения для обработки и потоковой передачи медиаконтента в ОС Linux»

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: Специалист по информационным системам

Очное отделение, курс IV, группа ИПО-41.22

Обучающийся: Демидов Дмитрий Вячеславович

Руководитель: Рыбаков Александр Сергеевич

Выпускная квалификационная работа защищена с оценкой « ____ » _____

Председатель ГЭК _____ / _____

Москва, 2026 г.

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
Государственное бюджетное профессиональное образовательное учреждение города
Москвы «Колледж малого бизнеса № 4»
(ГБПОУ КМБ № 4)

РАССМОТРЕНО И СОГЛАСОВАНО
Протокол № _____
от « ____ » _____ 2026 г.
ПЦК Информационных технологий
_____ / А.С. Чепрасова

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

Специальность: 09.02.07 Информационные системы и программирование
Обучающегося: Демидов Дмитрий Вячеславович
(Ф.И.О. полностью)
Очное отделение, курс IV, группа ИПО-41.22
Тема: Разработка и оптимизация высоконагруженного приложения для
обработки и потоковой передачи медиаконтента в ОС Linux

утверждена приказом по образовательной организации № **ОД-422** от **14.04.2026** г.

Срок сдачи студентом законченной работы **18.05.2026** г.

Основные вопросы, подлежащие разработке (исследованию), краткое содержание ВКР:

1. Введение – актуальность, цель, задачи, объект и предмет исследования, методы, ожидаемые результаты.

2. Аналитическая часть:

3. Практическая часть:

4. Заключение – итоговые выводы, достижения цели, экономическая и практическая эффективность, рекомендации по внедрению.

5. Список литературы: ≥ 35 источников, из них $\geq 60\%$ — за последние 5 лет; нормативные документы, ГОСТ, ISO/IEC, статьи журналов, профильные ресурсы.

6. Приложения:

Руководитель ВКР: _____ / Рыбаков А. С.
(подпись) (расшифровка)

Дата выдачи задания на ВКР: **«15» апреля 2026** г.

Задание принял к исполнению:

Демидов Д. В. / _____ / **«15» апреля 2026** г.
(Фамилия И.О. студента) (подпись)

Тема: «Разработка и оптимизация высоконагруженного приложения для обработки и потоковой передачи медиаконтента в ОС Linux»

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ.....	4
ВВЕДЕНИЕ.....	6
ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	10
1.1. Анализ предметной области.....	10
1.2. Анализ проблематики обеспечения производительности в высоконагруженных системах.....	12
1.3. Анализ специфики работы с медиаконтентом.....	14
1.4. Обоснование разработки собственного программного комплекса.....	17
1.5. Формализация требований к системе.....	19
Выводы по главе 1.....	29
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ.....	31
2.1. Проектирование архитектуры системы.....	31
2.2. Проектирование базы данных.....	39
2.3. Обоснование выбора технологий.....	44
2.4. Реализация функциональных модулей.....	47
2.5. Тестирование и обеспечение качества.....	54
2.6. Руководство пользователя.....	58
Выводы по главе 2.....	61
ЗАКЛЮЧЕНИЕ.....	63
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	66
ПРИЛОЖЕНИЯ.....	70
Приложение А.....	70
Приложение Б.....	75
Приложение В.....	83
Приложение Г.....	103
Приложение Д.....	106

СПИСОК СОКРАЩЕНИЙ

CDN — Content Delivery Network (сеть доставки контента);

RAM — Random Access Memory (оперативное запоминающее устройство);

ИС — Информационная система;

БД — База данных;

СУБД — Система управления базами данных;

API — Application Programming Interface (программный интерфейс приложения);

UI — User interface (пользовательский интерфейс);

ОС — Операционная система;

QoS — Quality of Service (качество обслуживания);

SQL — Structured Query Language (язык структурированных запросов);

UML — Unified Modeling Language (унифицированный язык моделирования);

IDEF0 — Integration Definition for Function Modeling (нотация функционального моделирования);

SWOT — Strengths, Weaknesses, Opportunities, Threats (анализ сильных и слабых сторон, возможностей и угроз);

HTTP — HyperText Transfer Protocol (протокол передачи гипертекста);

HLS — HTTP Live Streaming (прямая HTTP-трансляция);

DASH — Dynamic Adaptive Streaming over HTTP;

СПО — Среднее профессиональное образование;

ПО — Программное обеспечение;

ВКР — Выпускная квалификационная работа;

OTT — Over-the-Top;

VOD — Video On Demand (Видео по запросу);

QoE — пользовательский опыт;

ABR — Adaptive bitrate streaming;

MPD — Media presentation description;

URI-адрес — Uniform resource identifier (Унифицированный локатор ресурса);

FPS — Frames per seconds (Кадры в секунду);

NVMe — Non-Volatile Memory Express;

TTFB — Time to first frame (Время до первого кадра);

ВВЕДЕНИЕ

Актуальность разработки

В последние годы наблюдается стремительный рост объемов цифрового видеотрафика в глобальной сети Интернет. Современные мультимедийные платформы и онлайн-кинотеатры сталкиваются со стабильно растущим потоком пользователей, что выдвигает жесткие требования к производительности серверной и клиентской инфраструктуры. Особую нишу занимают специализированные медиаплатформы, в частности сервисы дистрибуции медиаконтента, характеризующиеся специфической структурой трафика: резкими пиковыми нагрузками («эффект наплыва»), необходимостью поддержки множественных аудиодорожек локализации и сложной графической обработки на стороне клиента.

Разработка систем, способных обрабатывать тысячи одновременных запросов на потоковую передачу данных без деградации качества обслуживания (QoS), является нетривиальной инженерной задачей. Эффективное решение этой проблемы лежит на стыке программно-аппаратной оптимизации операционных систем семейства Linux, проектирования распределенных архитектур доставки контента (CDN) и создания эффективных, легковесных нативных клиентских приложений, способных использовать аппаратное ускорение для декодирования видео. Подобный комплексный подход позволяет существенно снизить издержки на содержание серверных мощностей и обеспечить высокую отзывчивость интерфейса для конечного пользователя.

Актуальность темы выпускной квалификационной работы обусловлена ростом объемов видеотрафика, и связанным с ним увеличением нагрузки на инфраструктуру, а так же отсутствием современного клиентского ПО для обработки и воспроизведения медиаконтента в ОС Linux.

Таким образом, возникает потребность в снижении издержек на серверную инфраструктуру в условиях высоких нагрузок, а так же разработке и оптимизации прототипа программного комплекса для эффективного трансляции медиаконтента целевой аудитории.

Объект и предмет исследования

Объектом исследования являются процессы обработки, кодирования и потоковой передачи медиаконтента в сетях доставки данных (CDN).

Предметом исследования являются методы оптимизации производительности инфраструктуры, и ПО для ОС Linux для минимизации задержек и потребления ресурсов.

Цель работы

Целью выпускной квалификационной работы является разработка и оптимизация прототипа программного комплекса для эффективной стриминга медиаконтента целевой аудитории.

Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести сравнительный анализ современных протоколов адаптивного вещания (HLS, DASH) и выявить специфику сетевой нагрузки при трансляции специализированного медиаконтента.
2. Спроектировать масштабируемую архитектуру серверной части, включающую модули балансировки нагрузки, транскодирования, двухуровневого кэширования и управления метаданными.

3. Реализовать и оптимизировать серверные компоненты системы с применением механизмов неблокирующего асинхронного ввода-вывода и низкоуровневой настройки сетевого стека ядра Linux.
4. Разработать нативное клиентское приложение для ОС Linux на базе языка программирования Python и графического стека GTK, реализующее интерфейс пользователя (UI) и эффективное воспроизведение потокового видео.
5. Интегрировать мультимедийный движок на базе библиотеки GStreamer в клиентское приложение с поддержкой аппаратного ускорения декодирования видеопотока.
6. Провести нагрузочное тестирование разработанного комплекса, проанализировать метрики утилизации системных ресурсов (CPU, RAM, дисков, сети) и оценить эффективность примененных методов оптимизации.

Теоретическая основа

Теоретической и методологической основой исследования служат: Федеральный закон «Об информации, информационных технологиях и о защите информации», ФГОС СПО по специальности 09.02.07 [2], стандарты серии ГОСТ 34 (на техническое задание и техническую документацию) [10], ГОСТ 19 [8] (Единая система программной документации), международные стандарты ИСО/МЭК 25010 (Качество программного обеспечения) [12], ИСО/МЭК 14496-10 (Кодирование аудиовизуальных объектов), ИСО/МЭК 23009 (Динамическая адаптивная потоковая передача данных по протоколу HTTP), ИСО/МЭК 12207 (Процессы жизненного цикла) [11], ИСО/МЭК 27001 (Информационная безопасность) [13], а также труды отечественных и зарубежных авторов в области интеграции API, архитектуры микросервисов и кодирования медиафайлов.[23],[24],[25],[26],[27].

Методы исследования

В процессе выполнения работы использовались следующие методы: анализ предметной области, изучение технической литературы, стандартов, статистический анализ данных, сравнительный анализ, объектно-ориентированное проектирование, функциональное моделирование, моделирование структуры данных, программная реализация и функциональное тестирование.

Ожидаемые результаты и области применения

В рамках выполнения ВКР планируется разработать прототип программного комплекса для эффективной передачи медиаконтента, обеспечить эффективность его работы.

Практическая значимость работы заключается в том, что разработанная архитектура, ПО и предложенные конфигурации оптимизации могут быть использованы для развертывания экономически эффективных и отказоустойчивых платформ на базе свободного программного обеспечения.

Результаты работы могут быть внедрены в деятельность онлайн-кинотеатров, других коммерческих организаций занимающихся потоковым интернет-вещанием, организацией средств доставки контента или созданием серверной инфраструктуры на базе ОС Linux.

ГЛАВА 1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Анализ предметной области

1.1.1 Сравнительный анализ современных протоколов потокового вещания медиаконтента

В современной индустрии веб-разработки и мультимедийных технологий стандартом де-факто для доставки видеоданных конечному пользователю является концепция адаптивного стриминга по протоколу HTTP с использованием ABR. Суть данной методологии заключается в декомпозиции исходного медиапотока на дискретные временные сегменты (чанки) фиксированной длительности с последующим их кодированием с различными параметрами битрейта и разрешения. Клиентское программное обеспечение, динамически оценивая текущую пропускную способность сетевого канала, запрашивает фрагменты соответствующего качества, предотвращая тем самым буферизацию и деградацию пользовательского опыта (QoE).

На сегодняшний день на рынке доминируют два открытых стандарта адаптивного вещания: HLS, разработанный корпорацией Apple, и международный стандарт DASH, стандартизированный консорциумом ИСО/МЭК.

Для обоснования архитектурного выбора в рамках разрабатываемой системы необходимо провести их глубокий технический сравнительный анализ:

1. Структура дескрипторов:

- HLS использует в качестве индексных файлов текстовые плейлисты формата расширенного M3U (файлы с расширением .m3u8). Структура представляет собой последовательный список URI-адресов видеосегментов с метатегами разметки. Данный формат обладает низкой избыточностью, легко парсится стандартными средствами и человекочитаем.
- DASH базируется на описании представления мультимедиа (MPD), имеющем структуру XML-документа. Формат .mpd является избыточным с точки зрения объема передаваемых служебных данных, однако предоставляет более широкие возможности для описания сложных иерархических структур данных.

2. Форматы транспортных контейнеров:

- Исторически HLS функционировал исключительно на базе контейнеров MPEG-2 TS (.ts), что создавало дополнительные потери производительности (около 10\% объема файла) из-за фиксированного размера пакетов в 188 байт и дублирования заголовков. В современных спецификациях добавлена поддержка фрагментированного MP4, что сблизило его с DASH.
- DASH изначально проектировался как независимый от контейнера протокол, преимущественно ориентированный на MP4 и WebM, что обеспечивает высокую эффективность сжатия метаданных на транспортном уровне.

3. Кэширование на стороне прокси-серверов:

Этот критерий стал определяющим для архитектуры высоконагруженного приложения. Из-за простоты структуры .m3u8 плейлистов и линейной структуры адресации чанков в HLS, промежуточные серверы кэширования (обратные прокси, такие как Nginx или распределенные узлы CDN) могут эффективно обрабатывать и удерживать статические сегменты видео в оперативной памяти (RAM). Запрос к HLS-серверу сводится к простому GET-запросу статического файла. В случае с DASH, из-за возможности запросов поддиапазонов байт из одного монолитного файла, эффективная конфигурация кэширования на уровне оперативной памяти требует значительно более сложных алгоритмов конфигурации прокси-серверов и увеличивает накладные расходы на вычисление смещений в памяти.

Для минимизации накладных расходов серверной инфраструктуры, упрощения логики распределения контента и обеспечения максимальной совместимости, в качестве базового протокола вещания в разрабатываемом программном комплексе выбран HLS.

1.2. Анализ проблематики обеспечения производительности в высоконагруженных системах

Проектирование систем с высоким классом нагрузки в сфере доставки медиаконтента сопряжено с преодолением ряда критических ограничений физического и программного характера. Для иллюстрации возникающих проблем рассмотрим расчетный сценарий функционирования системы, при котором репрезентативная группа пользователей объемом $N = 10\ 000$ субъектов одновременно инициирует сессию воспроизведения одного и того же медиаресурса.

1.2.1. Пропускная способность сети

Средний битрейт качественного видеопотока в разрешении Full HD (1080p, 60 FPS, кодек H.264) составляет порядка $R = 5\text{ Мбит/с}$. При одновременном обращении $N = 10\ 000$ пользователей суммарная пиковая пропускная способность сетевого интерфейса сервера должна составлять:

$$B = N \times R = 10000 \times 5\text{ Мбит/с} = 50\text{ Гбит/с}$$

Формула 1

Данный показатель превышает пропускную способность стандартных серверных сетевых интерфейсов. Отсюда возникает потребность не просто в мощном серверном оборудовании, а в разработке программной архитектуры, поддерживающей горизонтальное масштабирование и интеллектуальную балансировку трафика между независимыми серверами.

1.2.2. Ограничения дисковой подсистемы

Наиболее критичным «узким местом» монолитных и неоптимизированных систем является подсистема ввода-вывода жестких дисков. При просмотре видео 10 000 пользователей запрашивают разные части одного или разных файлов. Это порождает паттерн случайного чтения (Random Read).

Даже современные NVMe-накопители при высоком параллелизме запросов начинают испытывать задержки из-за переполнения очереди команд и задержек контроллера. Если медиасервер обращается к физическому диску за каждым чанком размером в 2–4 Мегабайта для каждого из 10000 пользователей, наступает деградация дисковой подсистемы, выражающаяся в лавинообразном росте показателя задержек. Серверные процессы переходят в состояние неблокируемого сна (состояние D в Linux), и система перестает отвечать на запросы.

Для преодоления данного ограничения разрабатываемое приложение должно реализовывать стратегию двухуровневого кэширования: преаллокацию такого контента в виртуальной файловой системе в оперативной памяти (Tmpfs) и агрессивное кэширование дескрипторов файлов.

1.2.3. Вычислительная нагрузка (CPU-bound задачи) при транскодировании

Процесс кодирования видеопотока на лету под различные разрешения силами центрального процессора (CPU) является вычислительно избыточным. Попытка медиасерверов (например, базовых конфигураций Plex или Jellyfin) транскодировать видеопоток индивидуально под технические ограничения каждого клиента при нагрузке в 10 000 RPS мгновенно приводит к утилизации CPU в 100% и отказу системы.

В рамках оптимизации высоконагруженной архитектуры необходимо разделение процессов: транскодирование должно осуществляться асинхронно (в фоновом режиме через очереди задач RabbitMQ/Celery средствами FFmpeg) на этапе публикации контента, а сервер вещания должен выполнять исключительно роль эффективного распределителя готовой статик.

1.2.4. Метрики эффективности системы

При оптимизации серверного ПО под управлением ОС Linux в рамках данной работы ключевое внимание уделяется трем метрикам:

- **Масштабируемость:**

Способность системы линейно увеличивать пропускную способность при добавлении новых аппаратных узлов без изменения кодовой базы приложения;

- **Отказоустойчивость:**

Способность комплекса сохранять работоспособность при частичной или полной неисправности некоторых узлов за счет применения балансировщиков нагрузки (Nginx) и механизмов автоматического перенаправления клиентских запросов на резервные серверы без прерывания видеопотока у пользователя;

- **Скорость отдачи первого кадра (TTFF):**

Временной интервал между кликом пользователя на кнопку воспроизведения и физическим отображением первого кадра на экране клиента. Оптимизация TTFF требует минимизации времени установления TLS-соединения, мгновенной отдачи компактного .m3u8 файла из оперативной памяти и минимизации времени инициализации медиадвижка на стороне клиента.

1.3. Анализ специфики работы с медиаконтентом

Специфика разрабатываемого онлайн-кинотеатра накладывает особые требования к подсистеме кодирования и декодирования данных. В отличие от реалистичных кинофильмов, анимационный контент характеризуется специфической визуальной структурой: преобладанием монотонных цветовых заливок, жестких геометрических контуров (линий) и резких переходов между статичными планами.

Данная специфика открывает возможности для оптимизации сжатия, но одновременно создает проблемы для стандартных кодеков.

1.3.1. Проблематика цветовой дискретизации и 10-битного видео

Современный видеоконтент высокого качества массово распространяется с глубиной цвета 10 бит на канал (вместо стандартных 8 бит). Использование 10-битного представления информации позволяет полностью ликвидировать так называемый «эффект бандинга» (ступенчатые градиенты на небе, дыме, источниках света), который крайне заметен на рисованных текстурах.

Однако стандартный и наиболее аппаратно совместимый кодек H.264 (AVC) в профиле *High 10 Profile (Hi10P)* практически не имеет поддержки аппаратного декодирования (Hardware Acceleration) на большинстве потребительских видеокарт. Это означает, что декодирование такого потока полностью ложится на CPU пользователя. При разработке нативного клиентского приложения на Python/GTK необходимо предусмотреть интеграцию низкоуровневых библиотек, способных эффективно распараллеливать декодирование 10-битного H.264, либо реализовывать на сервере автоматический конвейер перекодирования в более современные форматы.

1.3.2. Сравнительный анализ профилей сжатия (H.264, HEVC, AV1)

Для выбора целевых кодеков серверного конвейера обработки данных проведем анализ трех ключевых стандартов сжатия:

Таблица 1

H.264 / AVC	Базовая совместимость, высокий битрейт, поддержка большого количества устройств
HEVC / H.265	Экономия ~40% емкости, эффективная реализация градиентов. Требуется лицензий, частичная поддержка устройствами
AV1	Максимальное сжатие (~50% эффективнее H.264). Открытый, высокая нагрузка на кодирование

1. H.264 (AVC):

Обладает абсолютной аппаратной совместимостью на стороне любых клиентских устройств (включая старые системы под управлением Linux). Идеален как базовый профиль для обеспечения гарантированного воспроизведения, однако требует высоких затрат дискового пространства и пропускной способности сети;

2. HEVC (H.265):

Обеспечивает экономию до 40% битрейта по сравнению с H.264 при сохранении аналогичного визуального качества. Профиль *HEVC Main 10* нативно поддерживает 10-битное видео и отлично сжимает графику. Недостатком является патентная обремененность и отсутствие нативной поддержки в некоторых свободных дистрибутивах Linux «из коробки» без установки проприетарных кодеков;

3. AV1 (AOMedia Video 1):

Новейший открытый лицензионно-свободный кодек. Демонстрирует превосходные результаты сжатия плоских текстур и линий. Экономия полосы пропускания достигает 50\% относительно H.264. Главный минус это экстремально высокая вычислительная сложность кодирования на стороне сервера, что накладывает жесткие требования к производительности фонового серверного транскодера.

1.4. Обоснование разработки собственного программного комплекса

При постановке задачи разработки возникает вопрос об экономической и инженерной целесообразности создания собственного программного продукта при наличии развитых готовых решений с открытым исходным кодом (Plex, Jellyfin, Emby, nginx-rtmp-module).

Однако детальный анализ их архитектурных особенностей показывает невозможность их эффективного применения в рамках заявленной темы выпускной квалификационной работы:

1. Монолитность архитектуры существующих медиасерверов:

Такие решения, как Jellyfin, спроектированы по принципу монолита «всё в одном». База данных (обычно SQLite), модуль авторизации, парсер метаданных и стриминг-движок выполняются в рамках одного системного процесса. В условиях Highload-нагрузки ошибка модуля авторизации из-за большого потока пользователей приводит к полному отказу стриминговой составляющей, что нарушает принцип отказоустойчивости. Разрабатываемое решение базируется на микросервисной архитектуре, разделяющей слой хранения метаданных (PostgreSQL/Redis), бизнес-логику API и легковесные Stateless-узлы раздачи HLS.

2. Неэффективное использование ресурсов ОС Linux готовыми решениями:

Популярные платформы используют высокоуровневые среды выполнения (.NET Core у Jellyfin), которые обладают избыточным потреблением оперативной памяти и не позволяют осуществлять тонкую настройку системных вызовов ядра Linux. Собственная реализация бэкенда на компилируемых или асинхронных языках (Go, Rust или асинхронный Python/FastAPI) в связке с Nginx позволяет использовать механизмы *epoll* и *io_uring* для неблокирующей обработки сетевых сокетов, минимизируя накладные расходы на переключение контекста процессора (Context Switch).

3. Отсутствие специализированных десктопных клиентов для экосистемы Linux:

Существующие платформы ориентированы либо на веб-интерфейс, либо на мобильные ОС и SmartTV. Веб-интерфейсы в среде Linux не имеют прямого низкоуровневого доступа к подсистеме VA-API (Video Acceleration API) для всех типов видеокарт, что приводит к перегреву CPU клиентских машин и пропуску кадров при воспроизведении тяжелого 10-битного контента. Разрабатываемое в рамках проекта нативное приложение на Python и GTK (PyGObject) позволяет напрямую интегрировать пайплайн мультимедийного фреймворка Gstreamer в оконную систему Wayland, обеспечивая аппаратное декодирование видеопотока на уровне операционной системы, стабильные 60 FPS интерфейса за счет асинхронного выполнения сетевых запросов и корректный нативный рендеринг сложных векторных субтитров через *libass*.

1.5. Формализация требований к системе

1.5.1. Функциональные требования к системе

Функциональные требования описывают законченное поведение программного комплекса в ответ на действия пользователей или системные события. Учитывая двухуровневую архитектуру «клиент-сервер», декомпозиция функциональных требований производится отдельно для серверного и клиентского сегментов.

Серверный сегмент

Серверная часть программного комплекса обязана обеспечивать выполнение следующих групп функций:

1. Подсистема управления контентом и публикации:

- 1) Система должна обеспечивать автоматический прием исходных медиаконтейнеров (форматов MKV, MP4) высокой четкости.
- 2) Фоновый конвейер обработки обязан осуществлять валидацию входных файлов и их автоматическое транскодирование с помощью утилиты FFmpeg в фиксированный набор разрешений (целевой профиль: 1080p, 720p, 480p).
- 3) Система должна выполнять автоматическую сегментацию (нарезку) транскодированного видеопотока на дискретные чанки формата .ts длительностью 4 секунды и генерировать соответствующие индексные файлы (.m3u8) в соответствии со спецификацией **RFC 8216 (HLS)**.
- 4) Подсистема должна сопоставлять видеоряд со сторонними аудиодорожками локализации и файлами субтитров **(ASS/SSA)**, регистрируя связи в базе данных.

2. Подсистема авторизации и бизнес-логики (API Gateway):

- 1) Обеспечение регистрации и аутентификации пользователей с использованием протоколов безопасного обмена токенами.
- 2) Предоставление API-эндпоинтов для передачи клиенту структурированных метаданных (каталог контента, жанры, история просмотров конкретного пользователя, метки времени остановки плеера).

3. Подсистема высоконагруженного вещания:

- 1) Обратный прокси-сервер обязан обрабатывать входящие HTTP-запросы от клиентов на получение плейлистов и видеосегментов.
- 2) Система должна динамически проверять права доступа пользователя перед отдачей каждого индивидуального медиа-сегмента для предотвращения несанкционированного скачивания.

Клиентский сегмент

Нативное графическое приложение на Python в связке с GTK должно реализовывать следующий функционал:

1. Интерфейс навигации и аутентификации:

- 1) Отображение формы авторизации/регистрации и безопасное локальное сохранение сессионных токенов.
- 2) Рендеринг динамического каталога контента (постеры, описания, списки серий) в соответствии с рекомендациями **GNOME HIG**.
- 3) Реализация механизмов поиска и фильтрации элементов каталога без перезагрузки интерфейса.

2. Мультимедийный плеер:

- 1) Интеграция видеоокна, способного работать с *m3u8* плейлистами и воспроизводить адаптивный HLS-поток.
- 2) Предоставление интерфейса управления воспроизведением (Play, Pause, перемотка по таймлайну, изменение громкости).
- 3) Динамическое переключение аудиодорожек на лету без прерывания сессии вещания.
- 4) Нативный попиксельный рендеринг сложных векторных субтитров формата **ASS/SSA** поверх видеоряда с корректным отображением шрифтов и стилей оформления.

1.5.2. Нефункциональные требования к системе

Нефункциональные требования определяют качественные характеристики системы, такие как производительность, надежность, масштабируемость и безопасность в условиях повышенной эксплуатационной нагрузки.

Производительность и надежность (Highload-метрики)

Для обеспечения бесперебойного функционирования онлайн-кинотеатра устанавливаются следующие целевые метрики эффективности серверной части:

- 1) **Пропускная способность:** Архитектура сервера раздачи обязана выдерживать пиковую нагрузку не менее *10 000* одновременных запросов в секунду (**RPS**) на получение медиа-сегментов.
- 2) **Скорость отдачи первого кадра (TTFF):** Время с момента инициации запроса видеопотока клиентом до отображения первого кадра в интерфейсе плеера не должно превышать *1.5 секунды* при пропускной способности сети клиента от *15 Мбит/с*.
- 3) **Доступность системы:** Коэффициент доступности серверной инфраструктуры должен составлять не менее **99.9%**. При сбое

отдельной серверной единицы балансировщик нагрузки обязан перенаправить трафик на рабочий узел в течение времени 500 мс.

Масштабируемость

Архитектура стриминг-узлов должна быть полностью изолированной от состояния сессий (Stateless). Добавление новых серверов раздачи в кластер CDN должно приводить к линейному росту поддерживаемой аудитории без необходимости модификации кодовой базы серверной части.

Безопасность и защита данных

Хранение пользовательских паролей в базе данных должно осуществляться исключительно в виде хэшированных строк с использованием криптостойких алгоритмов, например, Argon2. Передача любых данных между клиентом и сервером должна быть инкапсулирована в зашифрованный протокол HTTPS TLS 1.3.

Система должна включать лимитирование частоты запросов к API на уровне балансировщика, чтобы предотвратить DoS-атаки на базу данных.

Требования к программно-аппаратному обеспечению

Для развертывания, эксплуатации и тестирования разработанного программного комплекса устанавливаются минимальные и рекомендуемые требования к программной и аппаратной среде.

В таблице ниже приведены регламентируемые требования к вычислительному узлу, выполняющему роль основного сервера обработки и дистрибуции контента.

Таблица 2

Компонент	Мин. требования	Реком. требования
Операционная система	Linux(Ubuntu 20.04 LTS)	Linux(Ubuntu 24.04 LTS)
Центральный процессор	4 ядра, x86_64	8 ядер, x86_64 или ARM64
Оперативная память	4 Гб DDR4	8 Гб DDR5
Дисковая подсистема	HDD-диск	NVMe PCIe 4.0
Сетевой интерфейс	40 Mbps	10/25 Gbps Ethernet

Выводы по главе 1

В главе проведёно исследование существующих технологических решений, анализа архитектурных проблем медиаплатформ и сформирован комплекс требований к разрабатываемому программному комплексу. Выявлены ключевые проблемы дистрибуции медиаконтента со специфическим паттерном потребления: возникновение кратковременных экстремальных пиковых нагрузок, деградация дискового ввода-вывода, не способность обеспечить требуемый уровень отказоустойчивости.

Проанализированы нормативно-правовые акты и стандарты программной инженерии (ИСО/МЭК 12207, 23009, 27001, ГОСТ 19, ГОСТ 34). Выбран технологический стек: Python, GTK, HLS.

Проведён сравнительный анализ существующих аналогов («МойСклад», SellerFox, СДЭК-Интеграция, калькулятор ФТС). Ни одно из решений не обеспечивает комплексной автоматизации маркетплейсов, логистики и таможи в единой платформе.

Сформулированы функциональные требования (6 модулей) и нефункциональные требования (производительность, надёжность, безопасность, удобство использования). Определены критерии оценки эффективности. Полученные результаты являются достаточной основой для перехода к практической разработке прототипа.

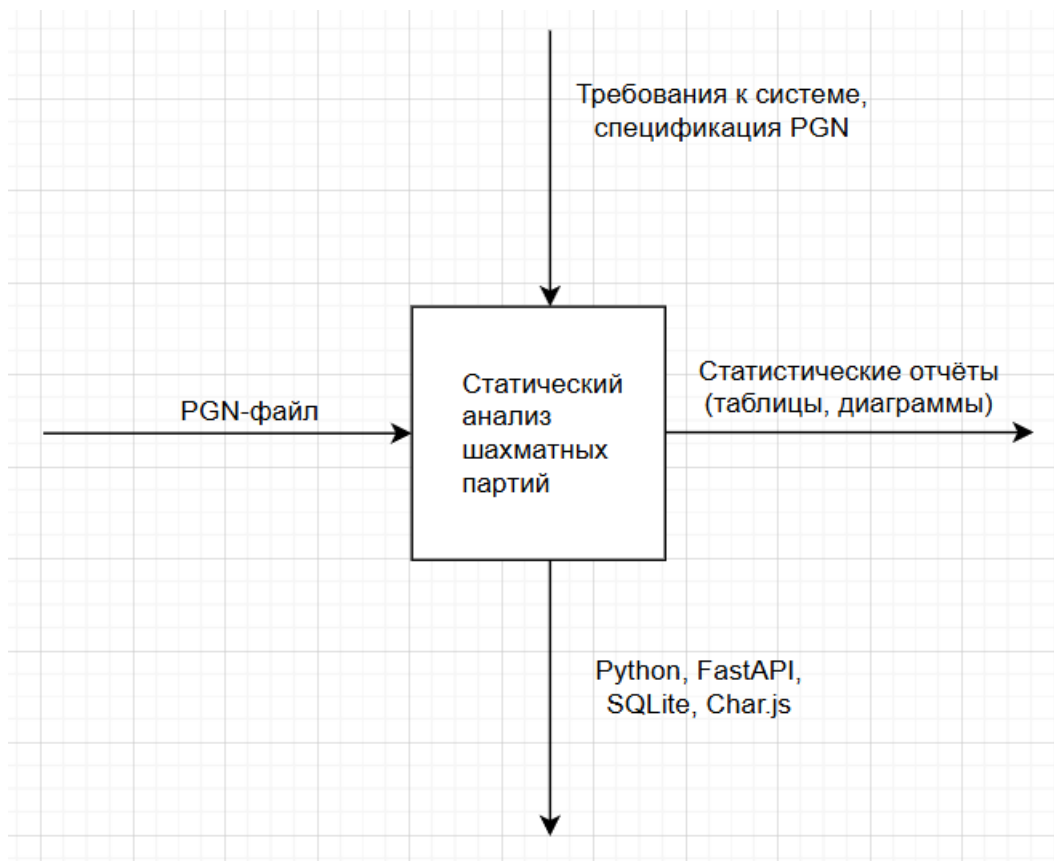


Рисунок 1 – Контекстная диаграмма IDEF0 (уровень A0)

Для более детального описания процессов была выполнена декомпозиция контекстного блока на четыре функциональных подблока:

- A1 — загрузка и валидация файла: пользователь отправляет файл через веб-форму, система проверяет его расширение и непустоту;
- A2 — парсинг PGN и извлечение данных: система последовательно считывает партии из файла и получает значения тегов заголовков;
- A3 — сохранение данных в базу: извлечённые метаданные записываются в таблицу базы данных;
- A4 — формирование статистики и визуализация: на основе сохранённых данных рассчитываются показатели и строятся диаграммы.

Диаграмма декомпозиции приведена на рисунке 2 (см. рисунок 2).

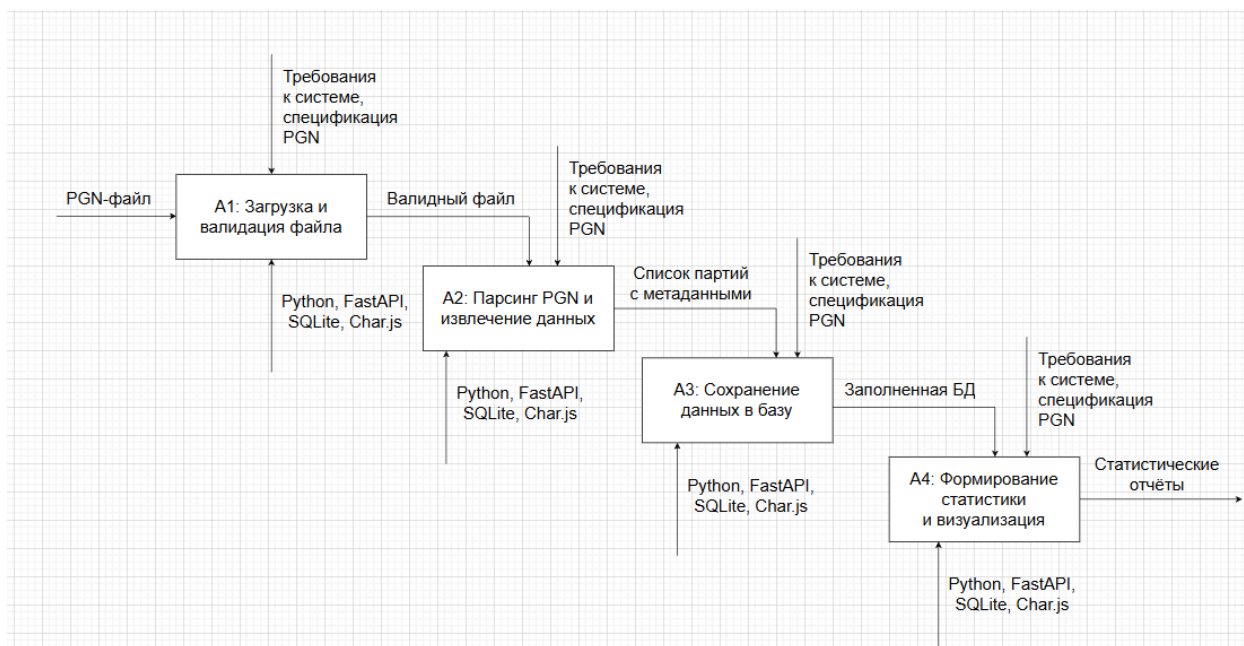


Рисунок 2 – Диаграмма декомпозиции IDEF0 (уровень A1)

1.2. Обоснование актуальности и постановка задачи

На протяжении последних лет наблюдается заметный рост интереса к шахматам. Отчасти это связано с появлением популярного медиаконтента, отчасти — с развитием крупных онлайн-платформ, таких как Lichess и Chess.com, которые сделали игру доступной для миллионов пользователей по всему миру. По данным самих платформ, ежедневно на них разыгрываются миллионы партий, и значительная их часть сохраняется в формате PGN.

Вместе с ростом количества сыгранных партий растёт и потребность в их осмыслении. Игрок, накопивший за несколько месяцев сотни записей, рано или поздно задаётся вопросами: какой процент партий заканчивается в его пользу, есть ли разница в результативности при игре белыми и чёрными, какие дебюты он применяет чаще всего и приносят ли они результат. Ответы на эти вопросы могут существенно помочь в совершенствовании игры, однако получить их вручную, просматривая файл за файлом, практически невозможно.

Казалось бы, для решения подобных задач существует достаточно инструментов. Однако при более внимательном рассмотрении выясняется,

что большинство из них ориентированы на другие цели. Онлайн-платформы в первую очередь предоставляют возможность играть и анализировать отдельные партии с помощью встроенных шахматных движков.

Профессиональные настольные программы, такие как ChessBase или SCID, предназначены для работы с большими базами данных и требуют определённого уровня подготовки. Для пользователя, который просто хочет загрузить свой файл с партиями и увидеть понятную сводку, подходящих решений оказывается немного.

Эта ситуация формирует проблему, на решение которой направлена настоящая работа. Суть проблемы можно сформулировать следующим образом: у шахматистов-любителей отсутствует простой и доступный инструмент, позволяющий быстро получить наглядную статистику по набору партий из PGN-файла без необходимости разбираться в сложном программном обеспечении.

Помимо практической значимости для конечного пользователя, данная тема представляет интерес и с образовательной точки зрения. Процесс создания подобной системы охватывает несколько ключевых компетенций, предусмотренных ФГОС по специальности 09.02.07: анализ предметной области, проектирование архитектуры, разработку серверной и клиентской частей веб-приложения, организацию хранения данных, визуализацию информации и тестирование.

Заказчиком разработки выступает ГБПОУ «Колледж малого бизнеса № 4». Практическая потребность заказчика состоит в получении учебного программного продукта, на примере которого можно продемонстрировать полный цикл создания информационной системы — от постановки задачи до готового работающего приложения.

С учётом вышесказанного можно выделить следующие факторы, определяющие актуальность работы:

- увеличение объёмов шахматных данных, доступных в цифровом виде, и потребность в их систематизации;
- недостаточная доступность простых инструментов статистического анализа PGN-файлов для широкой аудитории;
- возможность применения результатов работы как в практических, так и в учебных целях;
- соответствие темы профилю подготовки по специальности «Информационные системы и программирование».

Целью проекта является разработка веб-приложения, которое позволит пользователю загрузить PGN-файл, автоматически извлечь из него сведения о партиях и получить статистику в виде диаграмм и таблиц. Подробный перечень задач, необходимых для достижения этой цели, приведён во введении.

1.3. Анализ нормативно-методической базы

При разработке выпускной квалификационной работы недостаточно описать только предметную область и практическую задачу. Необходимо также определить нормативную и методическую основу, на которую опирается работа. Это важно по двум причинам. Во-первых, ВКР должна соответствовать установленным требованиям к содержанию и оформлению. Во-вторых, сам программный продукт должен разрабатываться с учётом общих принципов проектирования, документирования и оценки качества программного обеспечения.

Базовым документом, определяющим требования к подготовке выпускника по специальности 09.02.07 «Информационные системы и программирование», является федеральный государственный образовательный стандарт среднего профессионального образования. В данном стандарте зафиксированы компетенции, которыми должен обладать

выпускник: умение анализировать предметную область, проектировать программные решения, разрабатывать и интегрировать модули, использовать базы данных, выполнять тестирование и оформлять техническую документацию. Тема настоящей работы напрямую связана с указанными требованиями, поскольку включает и аналитическую, и проектную, и практическую составляющие.

При оформлении текста выпускной квалификационной работы основным документом выступает ГОСТ 7.32–2017, устанавливающий структуру и правила оформления отчётов о научно-исследовательской работе. Именно на его основе определяются требования к построению текста, оформлению разделов, таблиц, рисунков, формул, приложений и ссылок внутри документа. Несмотря на то, что ВКР не является научным отчётом в строгом смысле, данный стандарт используется как основа для приведения работы к единой форме.

Для оформления библиографического списка применяется ГОСТ Р 7.0.100–2018. Его использование необходимо для корректного описания книг, статей, нормативных документов и электронных ресурсов. Это особенно важно в дипломной работе, поскольку список литературы является обязательным элементом и должен быть составлен единообразно.

С точки зрения разработки самого программного продукта существенное значение имеет ГОСТ 34.602–2020, регулирующий содержание технического задания на создание автоматизированной системы. В рамках настоящей работы этот стандарт важен при подготовке приложения с техническим заданием, поскольку позволяет структурированно описать

назначение системы, требования к функциям, составу данных, условиям эксплуатации и критериям приёмки.

Кроме того, при составлении программной документации целесообразно учитывать ГОСТ 19.201–78, посвящённый требованиям к техническому заданию, а также ГОСТ 19.105–78 и ГОСТ 19.401–78, определяющие общие требования к программным документам и тексту программы. Эти стандарты не всегда используются в учебных работах в полном объёме, однако они задают понятную логику документирования программного продукта и делают описание разработки более профессиональным.

Отдельного внимания заслуживают международные стандарты, связанные с качеством и жизненным циклом программного обеспечения. В первую очередь это ISO/IEC 25010. Данный стандарт описывает характеристики качества программных систем: функциональную пригодность, надёжность, удобство использования, производительность, сопровождаемость, переносимость и ряд других параметров. Для разрабатываемой информационной системы наиболее значимыми являются функциональная пригодность, удобство использования, надёжность и производительность. Это объясняется тем, что пользователю важно быстро загрузить файл, получить корректный результат и без затруднений понять представленную статистику.

Не менее важен стандарт ISO/IEC 12207, посвящённый процессам жизненного цикла программного обеспечения. Его положения позволяют рассматривать разработку как последовательность этапов: анализ требований, проектирование, реализация, тестирование, внедрение и сопровождение. В

данной выпускной работе эта логика соблюдается: сначала определяется предметная область и формулируются требования, затем проектируется структура системы, после чего выполняется реализация, проверка и документирование результата.

Если рассматривать нормативно-методическую базу в прикладном аспекте, то можно выделить три уровня её использования в настоящей работе.

Первый уровень связан с оформлением самой ВКР. Здесь применяются требования методических рекомендаций колледжа, ГОСТ 7.32–2017 и ГОСТ Р 7.0.100–2018.

Второй уровень относится к разработке документации на программный продукт. На этом уровне используются ГОСТ 34.602–2020, ГОСТ 19.201–78, ГОСТ 19.105–78 и ГОСТ 19.401–78.

Третий уровень касается оценки качества создаваемой системы и логики её разработки. Здесь опорой выступают ISO/IEC 25010 и ISO/IEC 12207.

ГОСТЫ по шахматам где ?

Таким образом, проведённый анализ показывает, что нормативно-методическая база настоящей работы охватывает как требования к оформлению выпускной квалификационной работы, так и требования к

проектированию, документированию и оценке качества программного обеспечения. Это позволяет рассматривать разрабатываемую систему не просто как учебный пример, а как программный продукт, созданный с опорой на действующие стандарты и общепринятые подходы.

1.4. Анализ существующих решений и SWOT-анализ

Прежде чем переходить к проектированию собственной системы, необходимо рассмотреть уже существующие решения, которые используются для работы с шахматными партиями. Такой анализ позволяет понять, какие функции уже реализованы в популярных продуктах, какие подходы считаются удачными, а какие, наоборот, создают трудности для пользователя. Кроме того, сравнение с аналогами помогает обосновать, почему разрабатываемая система должна иметь именно тот набор функций, который был выбран в рамках данной работы.

Для анализа были выбраны четыре известных решения: Lichess.org, Chess.com, SCID и ChessBase (5). Эти продукты различаются по назначению, уровню сложности и целевой аудитории, поэтому позволяют достаточно полно рассмотреть существующий подход к работе с шахматными партиями.

Lichess.org является одной из самых популярных онлайн-платформ для игры в шахматы. Пользователь может играть партии в реальном времени, анализировать завершённые игры, просматривать варианты, использовать встроенный движок и изучать дебюты. Платформа поддерживает импорт партий в формате PGN и позволяет просматривать отдельные записи. Преимуществом данного решения являются бесплатная доступность, современный интерфейс и большое число встроенных инструментов. Однако Lichess в первую очередь ориентирован именно на игру и анализ конкретных партий, а не на статистическую обработку загруженного архива как единого

набора данных. Для пользователя, которому нужна простая сводка по большому числу партий, этот сервис оказывается не самым удобным.

Chess.com решает сходный круг задач, но делает больший акцент на экосистему: игра, обучение, решение задач, анализ партий, статистика активности, рейтинги и персональные отчёты. По функциональности это очень развитая платформа, однако часть возможностей предоставляется только в платной подписке. Кроме того, интерфейс Chess.com рассчитан на широкий набор сценариев, и поэтому пользователь, которому нужен исключительно импорт PGN и вывод базовой статистики, сталкивается с большим количеством лишних элементов.

SCID представляет собой специализированную настольную программу для работы с шахматными базами данных. В отличие от веб-сервисов, она ориентирована именно на хранение, поиск, фильтрацию и анализ массивов партий. Это делает её более близкой к задаче настоящей работы. Вместе с тем SCID рассчитан скорее на опытных пользователей. Интерфейс программы нельзя назвать интуитивно понятным для начинающего шахматиста, а сама работа требует установки приложения и освоения его внутренней логики.

ChessBase занимает особое место среди шахматных программ. Это профессиональный коммерческий продукт, ориентированный на тренеров, мастеров и серьёзно занимающихся игроков. Система обладает мощными средствами поиска, анализа, классификации партий и подготовки шахматного материала. При этом для обычного любителя ChessBase во многих случаях оказывается избыточным решением: цена высока, интерфейс достаточно сложен, а значительная часть функций в рамках повседневного пользовательского анализа просто не используется.

Сравнение показывает, что на рынке уже существуют как универсальные онлайн-платформы, так и специализированные программы для шахматного анализа. Тем не менее среди них трудно выделить решение, которое одновременно отвечало бы четырём условиям: простота использования, отсутствие необходимости в установке сложного ПО, ориентация на обработку пользовательского PGN-файла и наглядное представление базовой статистики.

Ниже приведено обобщённое сравнение рассмотренных решений.

Таблица 1.

Сравнение существующих решений для работы с шахматными партиями

Критерий	Lichess.org	Chess.com	SCID	ChessBase	Разрабатываемая система
Формат работы	Веб-сервис	Веб-сервис	Настольная программа	Настольная программа	Веб-приложение
Поддержка PGN	Да	Да	Да	Да	Да
Загрузка собственных партий	Да	Да	Да	Да	Да
Анализ набора партий как архива	Ограничено	Ограничено	Да	Да	Да
Построение простой статистики	Частично	Частично	Да	Да	Да
Понятность для любителя	Высокая	Средняя	Низкая	Низкая	Высокая
Необходимость установки	Нет	Нет	Да	Да	Нет
Стоимость	Бесплатно	Частично платно	Бесплатно	Платно	Бесплатно

На основе таблицы можно сделать несколько выводов. Во-первых, все рассмотренные решения поддерживают формат PGN, что подтверждает его распространённость и практическую значимость. Во-вторых, профессиональные продукты действительно предоставляют более широкий инструментарий, но расплачиваются за это сложностью интерфейса и, в ряде случаев, высокой стоимостью. В-третьих, веб-платформы удобны с точки зрения доступности, однако не всегда дают пользователю именно ту форму статистики, которая требуется при анализе собственного набора партий.

Таким образом, разрабатываемая система занимает промежуточную нишу. Она не стремится конкурировать с профессиональными шахматными базами по глубине анализа и не повторяет логику универсальных игровых платформ. Её задача уже и конкретнее: принять PGN-файл, выделить из него ключевые данные, сохранить их и представить пользователю в виде понятной статистики.

Для более полной оценки будущего проекта был выполнен SWOT-анализ. Такой подход позволяет рассмотреть систему не только с точки зрения её внутренних свойств, но и в контексте внешней среды.

Таблица 2.

SWOT-анализ разрабатываемой информационной системы

Сильные стороны (Strengths)	Слабые стороны (Weaknesses)
Простота использования; понятный веб-интерфейс; отсутствие необходимости в установке дополнительного шахматного ПО; автоматическая обработка нескольких партий из одного файла; наглядная визуализация результатов.	Ограниченный набор аналитических функций; отсутствие движкового анализа; зависимость качества статистики от полноты заполнения PGN-тегов; отсутствие многопользовательского режима.
Возможности (Opportunities)	Угрозы (Threats)
Расширение функциональности в будущем; добавление фильтрации по игрокам и дебютам; подключение расширенной аналитики; использование системы в учебных целях; интеграция с внешними шахматными сервисами.	Конкуренция со стороны крупных шахматных платформ; быстрое развитие существующих аналогов; изменение пользовательских ожиданий; ограниченность времени на развитие учебного проекта.

Результаты SWOT-анализа показывают, что основная ценность разрабатываемой системы заключается в её доступности и узкой специализации. Пользователь получает именно тот функционал, который нужен для быстрой статистической обработки собственных партий, без перегруженного интерфейса и сложной настройки. Одновременно анализ указывает и на ограничения проекта: система не предназначена для профессионального шахматного анализа и не заменяет крупные специализированные платформы.

Тем не менее в рамках поставленной задачи это не является недостатком в полном смысле слова. Напротив, отказ от избыточных функций позволяет сосредоточиться на понятной и полезной пользовательской задаче. Для учебного проекта такой подход можно считать оправданным, поскольку он обеспечивает баланс между практической значимостью, реализуемостью и качеством конечного результата.

1.5. Формализация требований к системе

На основании проведённого анализа предметной области и существующих решений можно перейти к формализации требований, которым должна соответствовать разрабатываемая информационная система. Этот этап важен, поскольку чёткое описание требований определяет границы проекта и задаёт конкретные критерии, по которым в дальнейшем можно будет оценить результат.

Требования к системе принято разделять на две группы: функциональные, описывающие, что именно система должна делать, и нефункциональные, определяющие, каким образом она должна это делать.

Функциональные требования

Первая группа требований связана с загрузкой данных. Система должна предоставлять пользователю веб-форму, с помощью которой можно выбрать файл с расширением .pgn и отправить его на сервер. При этом должна выполняться базовая проверка: файл не должен быть пустым, а его расширение должно соответствовать формату PGN. Система должна корректно обрабатывать файлы, содержащие как одну, так и несколько десятков или сотен партий.

Вторая группа требований касается обработки содержимого файла. После загрузки система должна автоматически разбирать текст и извлекать из каждой партии следующие данные:

- имя игрока, выступавшего белыми фигурами;
- имя игрока, выступавшего чёрными фигурами;
- результат партии;
- дату проведения;
- код дебюта по классификации ECO;
- название дебюта;

- количество ходов.

Если какой-либо из тегов в записи партии отсутствует, система не должна прерывать работу. Вместо этого соответствующее поле должно быть заполнено значением по умолчанию или оставлено пустым.

Третья группа требований относится к хранению данных. Извлечённые сведения должны сохраняться в реляционной базе данных. Структура хранения должна позволять выполнять выборки и агрегирование: подсчёт количества партий по результату, группировку по дебютам, получение полного списка записей.

Четвёртая группа требований описывает статистический и визуальный вывод. Система должна рассчитывать и отображать на странице следующие элементы:

- круговую диаграмму, показывающую соотношение побед белых, побед чёрных и ничьих;
- столбчатую диаграмму, сравнивающую количество побед белых и чёрных;
- таблицу с пятью наиболее часто встречающимися дебютами;
- полную таблицу всех загруженных партий с указанием игроков, результата, даты, дебюта и числа ходов.

Пятая группа требований касается управления данными. Перед загрузкой нового файла система должна автоматически очищать ранее сохранённые записи, чтобы результаты разных загрузок не смешивались. Кроме того, пользователю должна быть доступна возможность вручную очистить данные.

UML – прецендентов

Нефункциональные требования

Помимо функциональных возможностей, к системе предъявляется ряд требований, связанных с качеством её работы.

Удобство использования. Интерфейс должен быть организован таким образом, чтобы пользователю было достаточно выполнить минимальное количество действий для получения результата. Не должно требоваться предварительного обучения или изучения документации для базового сценария работы.

Надёжность. Система должна устойчиво работать при загрузке файлов с неполными данными, при отсутствии отдельных тегов в PGN-записях, а также при попытке загрузить файл неверного формата. Во всех подобных ситуациях пользователю должно выводиться понятное сообщение об ошибке вместо аварийного завершения работы.

Производительность. Обработка файла, содержащего до ста партий, должна выполняться за разумное время — не более нескольких секунд. При увеличении объёма данных допускается пропорциональный рост времени обработки.

Кроссплатформенность. Поскольку система реализуется как веб-приложение, она должна корректно работать в любом современном браузере. Установка дополнительного программного обеспечения на стороне пользователя не требуется.

Сопровождаемость. Код проекта должен быть структурирован по модулям, каждый из которых отвечает за свою задачу. Это упрощает внесение изменений, поиск ошибок и возможное расширение функциональности в будущем.

Метрики качества

Для оценки соответствия системы предъявленным требованиям были определены конкретные метрики, основанные на характеристиках качества из стандарта ISO/IEC 25010.

Таблица 3.

Метрики качества информационной системы

Характеристика качества	Метрика	Целевое значение
Функциональная пригодность	Доля реализованных функциональных требований	100 %
Надёжность	Число аварийных завершений при загрузке корректного файла	0
Удобство использования	Количество действий пользователя для получения результата	Не более 3
Производительность	Время обработки файла, содержащего 100 партий	Не более 5 секунд
Сопровождаемость	Разделение кода на независимые модули	Да

Пользовательский сценарий

Типовой сценарий взаимодействия с системой выглядит следующим образом. Пользователь открывает веб-приложение в браузере. На главной странице он видит форму для выбора файла. Нажав кнопку выбора, он указывает PGN-файл, хранящийся на его компьютере. После нажатия кнопки загрузки файл передаётся на сервер. Система считывает его содержимое, разбирает каждую найденную партию, извлекает нужные данные и сохраняет их. Затем на основе сохранённых данных формируется статистика, и пользователю отображается страница с диаграммами и таблицами. Если пользователь хочет проанализировать другой файл, он может загрузить новый — старые данные при этом будут заменены автоматически.

Ограничения

Для полноты описания следует указать ряд ограничений, принятых на этапе проектирования.

Система не предусматривает регистрацию и авторизацию пользователей. Любой человек, имеющий доступ к веб-приложению, может загружать файлы и просматривать результаты.

Система не использует шахматный движок для оценки качества ходов. Анализ ограничивается статистической обработкой метаданных, содержащихся в заголовках PGN-записей.

Данные хранятся в базе в рамках текущего сеанса работы. При загрузке нового файла ранее сохранённые записи удаляются. История загрузок не ведётся.

Эти ограничения являются осознанным выбором, обусловленным масштабом учебного проекта и стремлением сосредоточиться на основной задаче — корректной обработке PGN и наглядной визуализации результатов.

Все сформулированные функциональные и нефункциональные требования, метрики качества, ограничения и порядок приёмки были сведены в единый документ и оформлены в виде технического задания. Техническое задание на разработку информационной системы представлено в приложение А.

Выводы по главе 1

В первой главе были рассмотрены теоретические и аналитические основы разработки информационной системы для статистического анализа шахматных партий формата PGN.

В ходе анализа предметной области было установлено, что формат PGN является распространённым и удобным средством хранения шахматных партий в цифровом виде. Его структура позволяет выделять из записей не

только последовательность ходов, но и метаданные, представляющие интерес для последующей статистической обработки. Было также показано, что для пользователя значим не только просмотр отдельных партий, но и получение сводной информации по большому количеству игр.

При обосновании актуальности темы было выявлено противоречие между ростом числа доступных шахматных данных и недостатком простых инструментов, позволяющих анализировать пользовательские PGN-файлы без обращения к сложным профессиональным системам. Это подтвердило целесообразность разработки отдельного веб-приложения, ориентированного на шахматистов-любителей.

Проведённый анализ нормативно-методической базы показал, что разработка и оформление работы должны опираться на действующие государственные и международные стандарты, регулирующие структуру выпускной квалификационной работы, требования к программной документации и подходы к оценке качества программного обеспечения.

Сравнение существующих решений позволило установить, что современные шахматные сервисы и программы либо перегружены дополнительными возможностями, либо рассчитаны на более подготовленного пользователя. SWOT-анализ подтвердил, что разрабатываемая система имеет сильные стороны, связанные с простотой использования, наглядностью и узкой практической направленностью, при этом её ограничения являются допустимыми в рамках учебного проекта.

На заключительном этапе главы были сформулированы функциональные и нефункциональные требования к системе, определены основные метрики качества, пользовательский сценарий и ограничения проекта. Это позволило зафиксировать границы разработки и создать основу для перехода к проектированию и практической реализации информационной системы.

Таким образом, первая глава подготовила теоретическую и методическую базу для выполнения практической части выпускной квалификационной работы.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Проектирование архитектуры системы

После определения требований к разрабатываемой информационной системе необходимо перейти к проектированию её архитектуры. На данном этапе формируется общее представление о том, из каких компонентов будет состоять приложение, как эти компоненты будут взаимодействовать между собой и каким образом будет организован процесс обработки пользовательских данных.

С учётом поставленной задачи для проекта была выбрана клиент-серверная архитектура. Такой подход является наиболее естественным для веб-приложения, поскольку позволяет разделить пользовательский интерфейс, серверную логику и хранение данных. Пользователь работает с приложением через браузер, а основная обработка выполняется на стороне сервера. Это упрощает использование системы и не требует установки

дополнительного программного обеспечения на компьютере конечного пользователя.

Разрабатываемая система строится по модульному принципу. Каждый её компонент выполняет строго определённую функцию, что делает проект более понятным, упрощает тестирование и позволяет в дальнейшем при необходимости расширять возможности приложения. В архитектуре системы можно выделить четыре основных уровня: пользовательский интерфейс, серверную часть, модуль обработки PGN-файлов и уровень хранения данных.

На уровне пользовательского интерфейса располагается веб-страница, открываемая в браузере. Именно через неё пользователь взаимодействует с системой: выбирает файл, запускает анализ, просматривает статистику и при необходимости очищает данные. Интерфейс включает форму загрузки, сообщения об успешной обработке или ошибках, блок с диаграммами и таблицы с результатами анализа. Для пользователя этот уровень является единственной видимой частью системы.

Серверная часть отвечает за обработку HTTP-запросов, поступающих от клиента. В рамках данной работы сервер реализован на языке Python с использованием фреймворка FastAPI. Его задача состоит в том, чтобы принять загружаемый файл, передать его в модуль парсинга, организовать запись данных в базу, получить агрегированную статистику и затем вернуть пользователю HTML-страницу с результатами. Таким образом, сервер выполняет роль центрального управляющего звена.

Следующий уровень — модуль обработки PGN-файлов. Он получает текстовое содержимое файла и последовательно разбирает все партии, которые в нём содержатся. Из каждой партии извлекаются необходимые поля: имена игроков, результат, дата, код дебюта, название дебюта и количество ходов. Для повышения надёжности обработки используется специализированная библиотека `python-chess`, позволяющая работать с

форматом PGN без необходимости разбирать запись полностью вручную. Наличие отдельного модуля парсинга делает архитектуру чище и позволяет изолировать логику обработки шахматных данных от остального кода приложения.

Уровень хранения данных представлен базой SQLite. После разбора файла сведения о партиях заносятся в таблицу базы данных, а затем используются для формирования статистики. Такой подход выбран не случайно. Хотя объём данных в учебном проекте сравнительно небольшой, использование базы данных позволяет не только хранить записи в структурированном виде, но и выполнять выборки, группировку и подсчёты с помощью стандартных механизмов. Это делает систему ближе к реальным прикладным информационным решениям.

Общая архитектура системы представлена на рисунке 3.

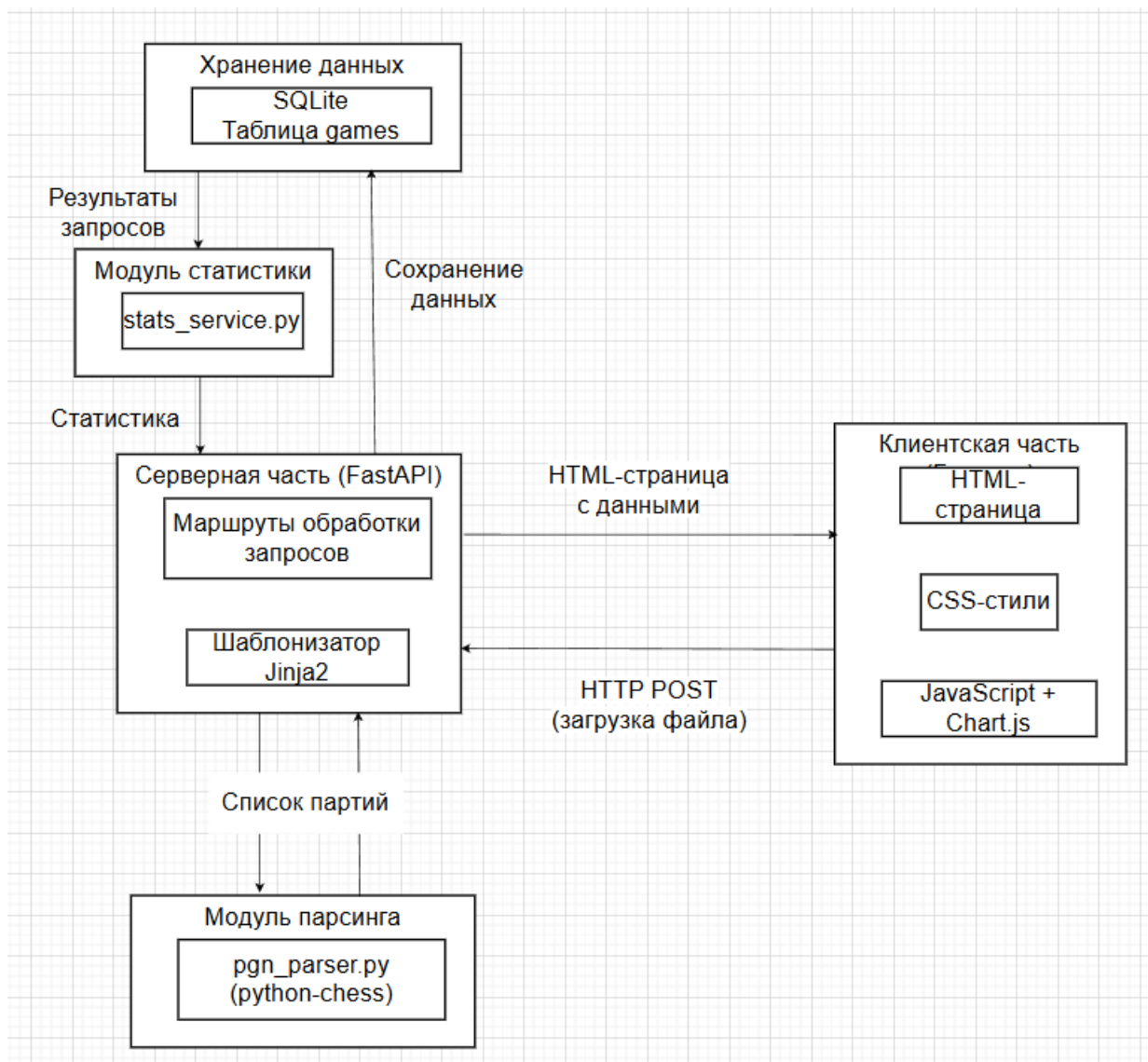


Рисунок 3 – Общая архитектура системы

В рамках данной архитектуры последовательность работы системы выглядит следующим образом. Пользователь открывает главную страницу приложения в браузере и выбирает файл формата PGN. После нажатия кнопки загрузки файл отправляется на сервер посредством HTTP POST-запроса. Сервер получает файл, выполняет его первичную валидацию, после чего передаёт содержимое в модуль парсинга. Модуль разбирает партии, формирует структурированный набор данных и возвращает его серверной части. Затем сервер сохраняет записи в базу данных, рассчитывает статистические показатели и передаёт их в HTML-шаблон. В результате в браузере отображается обновлённая страница с диаграммами и таблицами.

Подобная архитектура обладает несколькими важными преимуществами. Во-первых, она логично разделяет ответственность между отдельными частями системы. Во-вторых, каждый модуль может проверяться и дорабатываться независимо от остальных. В-третьих, при необходимости функциональность можно расширить без полной переработки проекта. Например, в будущем можно добавить фильтрацию по игрокам, экспорт отчётов или историю загрузок, не меняя базовый принцип взаимодействия компонентов.

Для формального описания взаимодействия пользователя с системой была построена диаграмма вариантов использования UML. На этой диаграмме представлен один основной актор — пользователь. Именно он инициирует все действия, связанные с загрузкой файла и просмотром результатов. В рамках текущей версии системы пользователь может выполнить следующие действия: загрузить PGN-файл, получить статистику по результатам партий, просмотреть диаграммы, ознакомиться с таблицей наиболее популярных дебютов, просмотреть полный список загруженных партий и очистить данные перед повторной загрузкой. Диаграмма вариантов использования приведена на рисунке 4.

Актёры:

- Не зарегистрированный пользователь
- Зарегистрированный пользователь
- Администратор турниров

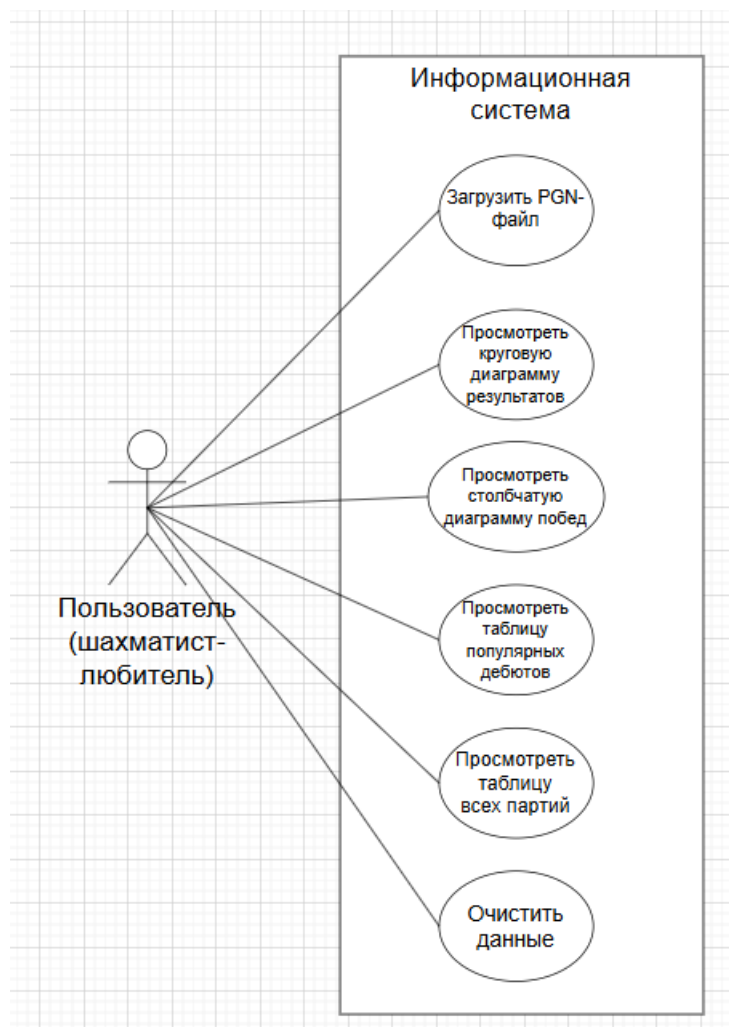


Рисунок 4 – Диаграмма вариантов использования

Диаграмма вариантов использования позволяет показать систему с точки зрения конечного пользователя. Это особенно важно для учебного проекта, поскольку она наглядно демонстрирует, какие функции реализованы в приложении и какие сценарии действительно поддерживаются. В данном случае набор вариантов использования намеренно ограничен только основными действиями, необходимыми для решения поставленной задачи. Отказ от второстепенных возможностей, таких как регистрация, авторизация или экспорт данных, был сделан осознанно, чтобы сосредоточиться на ключевом функционале.

Для описания внутренней логики работы системы была построена диаграмма активности UML. Она отражает последовательность действий,

которые выполняются после загрузки пользователем файла. Процесс начинается с выбора PGN-файла. Затем система проверяет расширение файла и его содержимое. Если файл не соответствует требованиям, пользователю выводится сообщение об ошибке, и выполнение сценария прекращается. Если файл корректен, выполняется чтение партий, извлечение метаданных, сохранение записей в базу данных, расчёт статистики и вывод результатов на экран. Диаграмма активности представлена на рисунке 5.

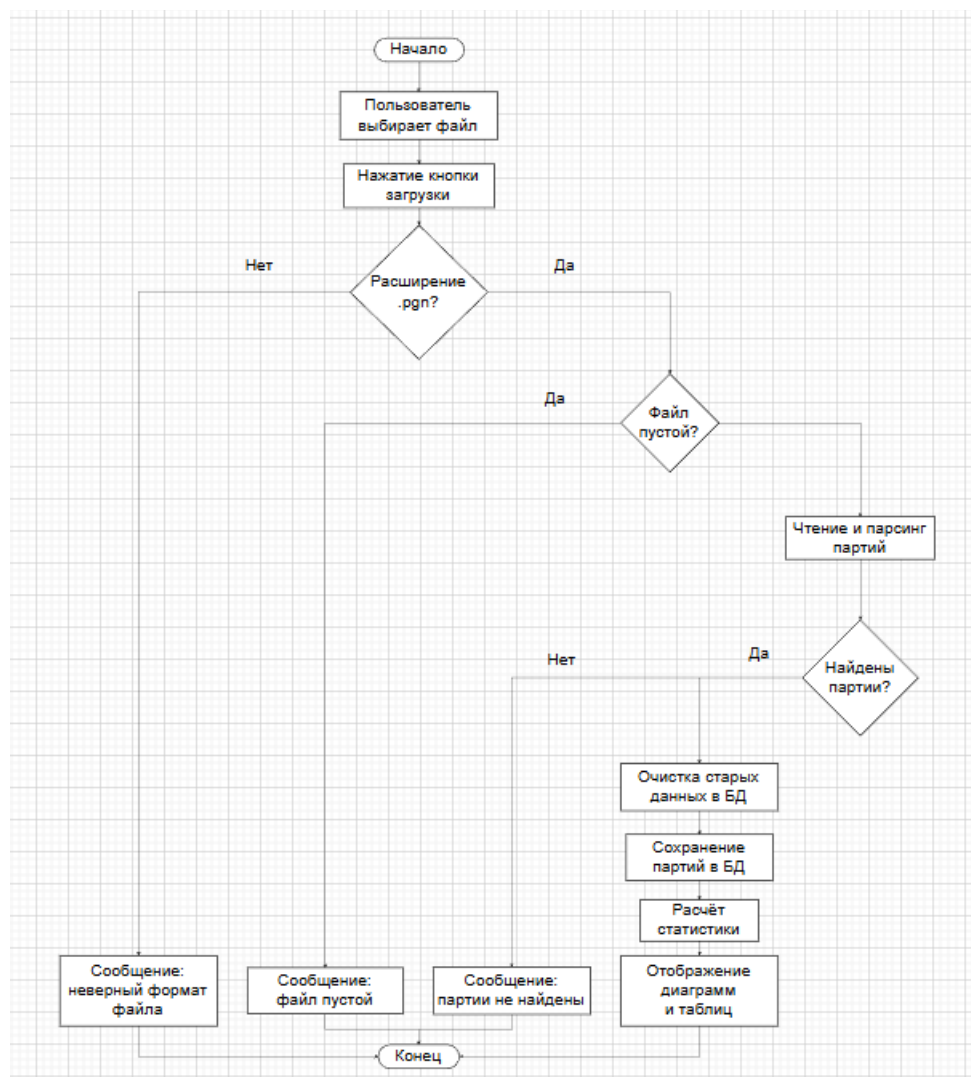


Рисунок 5 — Диаграмма активности процесса обработки PGN-файла

Диаграмма активности полезна тем, что показывает не просто состав функций, а сам алгоритм выполнения операций. Она позволяет отследить ветвления, связанные с проверкой корректности данных, и увидеть, как система переходит от одного шага к другому. В рассматриваемом проекте это особенно важно, поскольку обработка загружаемого файла включает как

успешный сценарий, так и возможные ошибки, например пустой файл или неверный формат.

Наконец, для описания взаимодействия отдельных компонентов системы была построена диаграмма последовательности UML. На ней отражено, как именно обмениваются сообщениями браузер, серверное приложение, модуль парсинга и база данных. Пользователь через браузер инициирует загрузку файла. Браузер отправляет запрос серверу. Сервер вызывает модуль парсинга, получает от него результаты, записывает данные в базу, извлекает статистику и формирует ответ. После этого браузер отображает готовую страницу. Диаграмма последовательности приведена на рисунке 6.

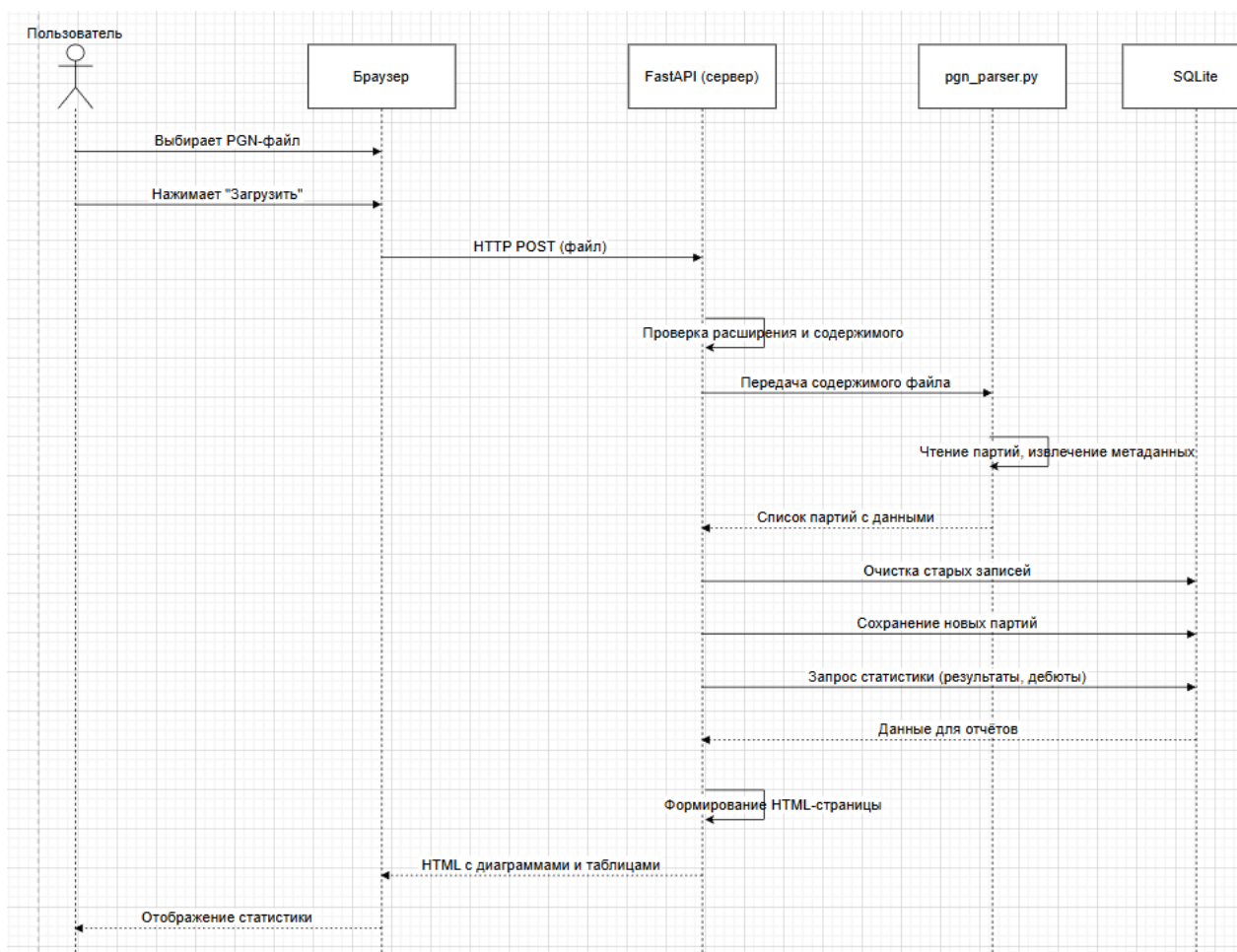


Рисунок 6 — **UML** Диаграмма последовательности взаимодействия компонентов системы

В отличие от диаграммы активности, диаграмма последовательности показывает не логику процесса как такового, а порядок взаимодействия между конкретными участниками. Для практической части дипломной работы это полезно тем, что позволяет обосновать выбранное распределение ролей между клиентом, сервером, модулем обработки и базой данных. Из диаграммы видно, что большая часть вычислительной и логической нагрузки сосредоточена на серверной стороне, тогда как браузер выполняет в основном функции ввода и отображения.

Таким образом, на этапе проектирования архитектуры была сформирована логическая модель разрабатываемой информационной системы. Выбранная клиент-серверная архитектура соответствует особенностям веб-приложения и обеспечивает достаточную простоту реализации при сохранении модульности. Определён состав основных компонентов, зафиксированы их функции и описаны сценарии взаимодействия. Полученные результаты создают основу для последующего проектирования базы данных и практической реализации системы.

Комплект построенных диаграмм, отражающих архитектуру системы, пользовательские сценарии и взаимодействие компонентов, приведён в приложении (см. Приложение Б).

2.2. Проектирование базы данных

На этапе проектирования базы данных необходимо определить, какие именно данные будут храниться в системе, каким образом они будут структурированы и какие средства будут использоваться для работы с ними. Поскольку разрабатываемая система ориентирована на обработку шахматных партий из PGN-файлов, основной единицей хранения является запись об одной партии с набором её характеристик.

Для хранения данных была выбрана СУБД SQLite. Этот выбор обусловлен рядом практических причин. SQLite не требует установки отдельного серверного приложения и работает как встраиваемая библиотека.

Вся база данных размещается в одном файле, который создаётся автоматически при первом запуске приложения. Это существенно упрощает развёртывание проекта: достаточно скопировать папку с программой на другой компьютер, и система будет работать без дополнительной настройки. Для учебного веб-приложения, не предполагающего параллельную работу множества пользователей, SQLite является оптимальным решением.

Для взаимодействия с базой данных из Python-кода используется библиотека SQLAlchemy. Она позволяет описывать структуру таблиц в виде классов на языке Python и выполнять операции с данными без необходимости писать SQL-запросы вручную. Такой подход принято называть ORM — Object-Relational Mapping. Использование ORM делает код более читаемым и снижает вероятность ошибок при формировании запросов.

На логическом уровне база данных содержит одну основную таблицу — games. В ней хранятся метаданные о каждой загруженной партии. Решение использовать единственную таблицу обусловлено тем, что система не предполагает разделения данных по пользователям, сессиям или турнирам. Все загруженные партии принадлежат одному набору, который формируется при каждой новой загрузке файла.

Структура таблицы games представлена ниже.

Таблица 4.

Структура таблицы games

Поле	Тип данных	Описание	Ограничения
id	INTEGER	Уникальный идентификатор записи	PRIMARY KEY, AUTOINCREMENT
white_player	TEXT	Имя игрока белыми	NOT NULL, по умолчанию «Не указано»
black_player	TEXT	Имя игрока чёрными	NOT NULL, по умолчанию «Не

			указано»
result	TEXT	Результат партии (1-0, 0-1, 1/2-1/2)	NOT NULL, по умолчанию «Не указано»
game_date	TEXT	Дата проведения партии	Может быть NULL
eco_code	TEXT	Код дебюта по классификации ECO	Может быть NULL
opening_name	TEXT	Название дебюта	Может быть NULL
move_count	INTEGER	Количество ходов в партии	Может быть NULL

Поле `id` является первичным ключом таблицы и автоматически увеличивается при добавлении каждой новой записи. Поля `white_player`, `black_player` и `result` отмечены как обязательные, поскольку без них запись о партии теряет практический смысл. Если в исходном PGN-файле эти теги отсутствуют, система подставляет значение по умолчанию. Остальные поля допускают пустые значения, так как не все PGN-записи содержат полный набор тегов. Например, в партиях, сыгранных в неформальной обстановке, может отсутствовать код дебюта или точная дата.

На физическом уровне таблица реализуется средствами SQLite. При первом запуске приложения файл базы данных создаётся автоматически в каталоге проекта. Индексирование по полю `id` выполняется автоматически благодаря ограничению PRIMARY KEY. Дополнительных индексов на данном этапе не предусмотрено, поскольку объём данных в рамках учебного проекта не требует оптимизации скорости выборки.

Для наглядного представления структуры данных была построена ER-диаграмма. Поскольку система работает с единственной таблицей и не предполагает связей между несколькими сущностями, диаграмма включает только одну сущность — Games — с перечисленными выше атрибутами. ER-диаграмма приведена на рисунке 7.

Games			
INTEGER	id	PK	Уникальный идентификатор
TEXT	white_player		Имя игрока белыми
TEXT	black_player		Имя игрока чёрными
TEXT	result		Результат партии
TEXT	game_date		Дата партии
TEXT	eco_code		Код дебюта ECO
TEXT	opening_name		Название дебюта
INTEGER	move_count		Количество ходов

Рисунок 7 - ER-диаграмма базы данных

Помимо ER-диаграммы, для описания программной структуры серверной части была построена диаграмма классов UML. Она отражает основные классы, используемые в проекте, их атрибуты и методы.

В системе можно выделить три ключевых класса:

Первый класс — Game. Он соответствует модели данных, описывающей таблицу games. Атрибуты этого класса повторяют поля таблицы: id, white_player, black_player, result, game_date, eco_code, opening_name, move_count.

Второй класс — PGNParser. Он отвечает за чтение PGN-файла и извлечение данных. Основной метод — parse_file, который принимает текстовое содержимое файла и возвращает список словарей с данными о партиях.

Третий класс — StatsService. Он формирует статистические показатели на основе данных из базы. Его методы включают: `get_results_distribution` — подсчёт побед и ничьих, `get_top_openings` — определение наиболее популярных дебютов, `get_all_games` — получение полного списка партий, `get_total_count` — подсчёт общего числа записей.

Диаграмма классов приведена на рисунке 8.

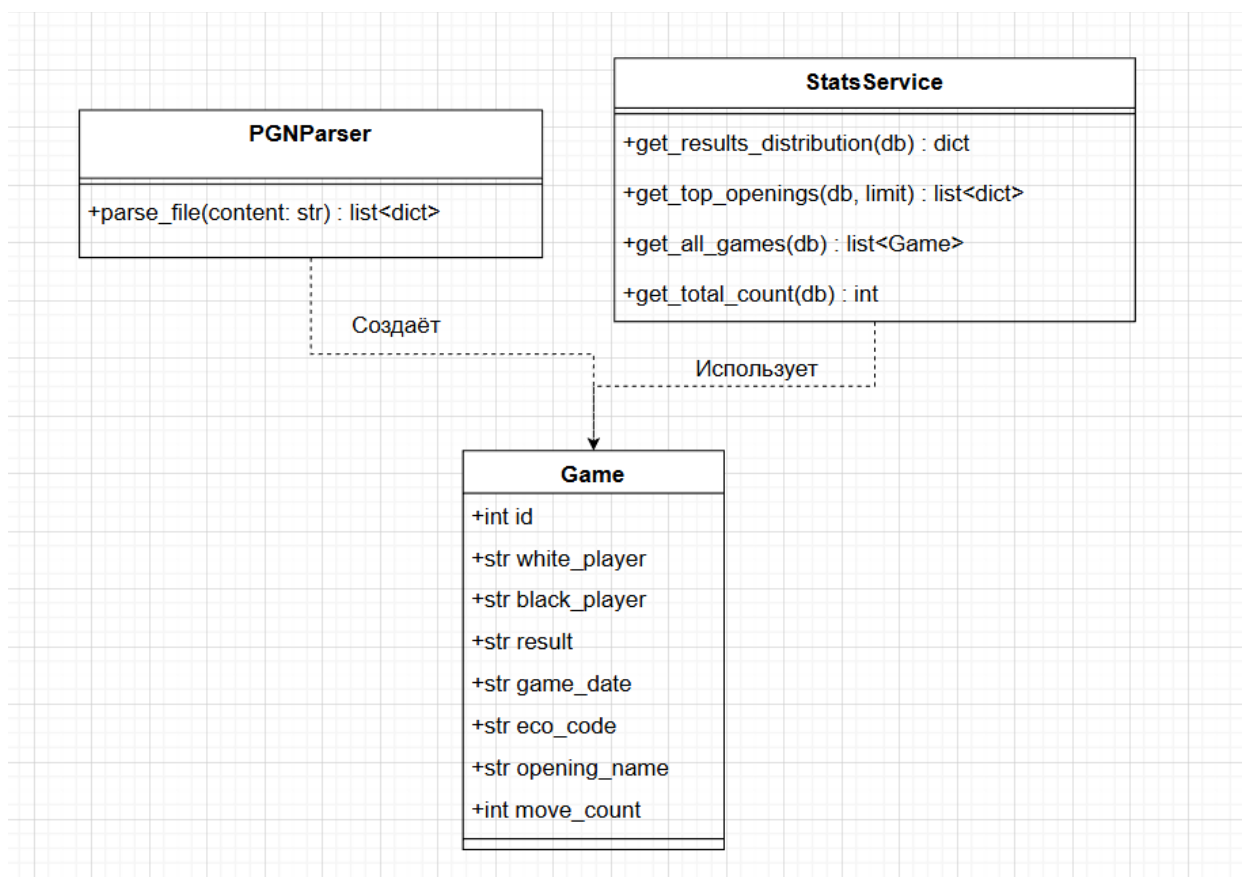


Рисунок 8 - Диаграмма классов UML

Выбранная структура базы данных является достаточной для решения поставленной задачи. Она позволяет хранить все необходимые метаданные, выполнять выборки по результатам и дебютам, а также формировать агрегированные показатели. При необходимости в дальнейшем структуру можно расширить, добавив, например, таблицу пользователей или таблицу загрузок, однако в рамках текущей версии проекта в этом нет необходимости.

ER-диаграмма базы данных и диаграмма классов, использованные при проектировании системы, приведены в приложении (см. Приложение Б).

2.3. Обоснование выбора технологий

После проектирования архитектуры и структуры хранения данных необходимо обосновать выбор конкретных технологий, на базе которых была реализована информационная система. Для учебного проекта важно не только добиться работоспособности приложения, но и подобрать такой стек, который позволит реализовать требуемый функционал в разумные сроки, сохранить понятную структуру кода и обеспечить удобство дальнейшего сопровождения.

При выборе технологий учитывались следующие факторы: простота освоения и использования, достаточность для поставленной задачи, наличие документации, доступность библиотек для работы с форматом PGN, а также возможность быстрого развёртывания приложения на локальном компьютере без сложной настройки.

В качестве основного языка программирования был выбран Python. Данный язык широко применяется в учебной и прикладной разработке благодаря лаконичному синтаксису, большому количеству готовых библиотек и удобству работы с текстовыми данными. Поскольку PGN-файл представляет собой текстовую структуру, Python хорошо подходит для решения задачи разбора и обработки его содержимого. Кроме того, язык обладает низким порогом вхождения и позволяет быстро разрабатывать прототипы и небольшие веб-приложения.

Для серверной части проекта был выбран фреймворк FastAPI. Его использование обусловлено тем, что данный фреймворк сочетает простоту, современный подход к построению веб-приложений и высокую скорость разработки. В отличие от более тяжёлых решений, таких как Django, FastAPI не перегружает проект дополнительными компонентами, которые в рамках данной работы не требуются. Для рассматриваемой информационной

системы нет необходимости в административной панели, системе авторизации, встроенной ORM или сложной маршрутизации. По этой причине применение FastAPI является более рациональным.

Ещё одним аргументом в пользу FastAPI является удобство запуска локального приложения. Сервер можно запустить одной командой, после чего он становится доступен в браузере. Это особенно важно в условиях учебного проекта, когда требуется быстро получить работающий результат и протестировать его на локальном компьютере.

Для формирования HTML-страниц используется шаблонизатор Jinja2. Он позволяет передавать данные из серверной части в HTML-шаблон и выводить их на странице с помощью простых конструкций. Благодаря этому статистические показатели, таблицы и сообщения могут формироваться динамически, без необходимости создавать полностью статические страницы. Использование Jinja2 делает интерфейс более гибким и удобно сочетается с FastAPI.

В качестве системы управления базой данных была выбрана SQLite. Это решение оказалось наиболее подходящим для проекта по нескольким причинам. Во-первых, SQLite не требует установки отдельного сервера и работает как встроенная база данных. Во-вторых, вся информация хранится в одном файле, что упрощает перенос приложения и резервное копирование. В-третьих, производительности SQLite достаточно для учебной системы, в которой предполагается работа с ограниченным объёмом данных. Выбор более тяжёлой СУБД, например PostgreSQL, в рамках данного проекта не дал бы существенных преимуществ, но потребовал бы дополнительной настройки.

Для взаимодействия с базой данных была использована библиотека SQLAlchemy. Она позволяет описывать структуру таблиц в виде Python-классов и выполнять операции добавления, удаления и выборки данных

средствами ORM. Такой подход повышает читаемость кода и делает работу с БД более структурированной. Кроме того, использование ORM позволяет проще соотносить программную модель системы с её логической структурой, описанной в диаграммах.

Отдельное внимание было уделено выбору средства для разбора PGN-файлов. Вместо ручного парсинга текста была использована библиотека `python-chess`. Это решение позволило сократить объём низкоуровневого кода и сосредоточиться на логике приложения. Библиотека поддерживает чтение PGN-записей, работу с заголовками партий и последовательностью ходов. Благодаря этому можно надёжно извлекать необходимые данные без необходимости самостоятельно разбирать каждую строку файла.

Для визуализации статистики была выбрана библиотека `Chart.js`. Она позволяет строить диаграммы непосредственно в браузере на основе данных, переданных с сервера. В рамках проекта потребовались два типа визуализации: круговая диаграмма распределения результатов и столбчатая диаграмма сравнения побед белых и чёрных. `Chart.js` хорошо подходит для таких задач, поскольку легко подключается к HTML-странице, не требует сложной настройки и обеспечивает наглядный внешний вид графиков.

Клиентская часть приложения реализована средствами HTML, CSS и JavaScript. HTML используется для формирования структуры страницы, CSS — для её визуального оформления, а JavaScript — для инициализации диаграмм. Такой набор технологий является стандартным для разработки веб-интерфейсов и полностью покрывает потребности настоящего проекта.

Итоговый стек технологий представлен в таблице 5.

Таблица 5.

Итоговый стек технологий проекта

Компонент	Выбранная технология	Назначение
Язык программирования	Python	Серверная логика, обработка данных
Веб-фреймворк	FastAPI	Обработка HTTP-запросов
Шаблонизатор	Jinja2	Формирование HTML-страниц
СУБД	SQLite	Хранение данных о партиях
ORM	SQLAlchemy	Работа с базой данных
Парсер PGN	python-chess	Чтение и разбор шахматных партий
Визуализация	Chart.js	Построение диаграмм
Клиентская часть	HTML, CSS, JavaScript	Интерфейс пользователя

Таким образом, выбранный стек технологий соответствует задачам проекта и обеспечивает разумный баланс между простотой разработки, функциональностью и удобством сопровождения. Все используемые средства являются доступными, хорошо документированными и достаточными для создания работающего веб-приложения, выполняющего статистический анализ шахматных партий формата PGN.

2.4. Реализация функциональных модулей

После завершения этапов проектирования архитектуры, базы данных и обоснования выбора технологий был выполнен переход к непосредственной программной реализации информационной системы. В данном разделе описывается структура проекта, назначение каждого модуля, логика их работы и результат, который видит пользователь при взаимодействии с приложением.

Структура проекта

Проект организован по модульному принципу. Каждый файл отвечает за отдельную область логики, что делает код более читаемым и облегчает его сопровождение. Ниже приведён перечень основных файлов проекта с указанием их назначения.

Файл `main.py` является точкой входа в приложение. В нём описаны маршруты обработки HTTP-запросов: отображение главной страницы, загрузка файла и очистка данных. Именно этот файл связывает между собой все остальные компоненты системы.

Файл `database.py` содержит настройки подключения к базе данных SQLite и создание сессий. В нём задаётся путь к файлу базы данных и описывается механизм получения подключения, который используется при каждом запросе.

Файл `models.py` описывает модель данных — класс `Game`, который соответствует таблице `games` в базе данных. Атрибуты класса повторяют поля таблицы: идентификатор, имена игроков, результат, дата, код дебюта, название дебюта и число ходов.

Файл `pgn_parser.py` содержит функцию разбора PGN-файлов. Он принимает текстовое содержимое загруженного файла, последовательно считывает партии и возвращает список словарей с извлечёнными данными.

Файл `stats_service.py` реализует функции формирования статистики. Он выполняет запросы к базе данных и возвращает распределение результатов, список популярных дебютов, полный перечень партий и общее число записей.

Папка `templates` содержит HTML-шаблон главной страницы — файл `index.html`. Именно этот шаблон формирует то, что видит пользователь в браузере: форму загрузки, сообщения, диаграммы и таблицы.

Папка `static` содержит файл `style.css`, определяющий внешний вид страницы: цвета, отступы, размеры элементов, оформление таблиц и кнопок.

Модуль загрузки файла

Загрузка файла осуществляется через HTML-форму на главной странице. Пользователь нажимает кнопку выбора файла, указывает PGN-

документ на своём компьютере и нажимает кнопку «Загрузить и проанализировать». После этого браузер отправляет файл на сервер в формате HTTP POST-запроса.

На стороне сервера выполняется несколько проверок. Прежде всего проверяется расширение файла: система принимает только файлы с расширением .pgn. Затем проверяется, не является ли файл пустым. Если хотя бы одна из проверок не пройдена, пользователю возвращается страница с сообщением об ошибке. Это позволяет избежать обработки заведомо некорректных данных и делает работу системы более устойчивой.

В случае успешной проверки содержимое файла декодируется из байтового представления в текстовую строку и передаётся в модуль парсинга.

Модуль парсинга

Модуль парсинга является ключевым компонентом системы, поскольку именно он отвечает за извлечение полезной информации из загруженного файла. Его работу можно описать следующим образом.

На вход поступает строка с полным текстовым содержимым PGN-файла. Эта строка преобразуется во внутренний текстовый поток, после чего начинается последовательное чтение партий. Для каждой найденной партии библиотека python-chess предоставляет доступ к заголовкам — тегам, записанным в квадратных скобках. Из них извлекаются значения полей White, Black, Result, Date, ECO и Opening.

Помимо заголовков, модуль подсчитывает количество ходов в партии. Для этого он проходит по основной линии игры и считает каждый выполненный ход. Полученное значение сохраняется вместе с остальными данными.

Если в записи какой-либо партии отсутствует один из тегов, модуль не прерывает работу. Вместо этого он подставляет значение по умолчанию или оставляет поле пустым. Такой подход обеспечивает устойчивость системы к

неполным данным, что особенно важно при обработке реальных PGN-файлов, качество заполнения которых может существенно различаться.

После обработки всех партий модуль возвращает список словарей, каждый из которых содержит данные одной партии. Этот список передаётся обратно в серверную часть для дальнейшего сохранения.

Модуль хранения данных

Перед записью нового набора партий система выполняет очистку таблицы games. Это сделано для того, чтобы результаты разных загрузок не смешивались. Пользователь анализирует один файл за раз, и статистика должна отражать содержимое именно последнего загруженного набора.

После очистки каждая партия из полученного списка записывается в таблицу как отдельная строка. Для этого создаётся экземпляр класса Game с соответствующими значениями полей, после чего он добавляется в текущую сессию базы данных. По завершении цикла записи выполняется фиксация изменений.

Модуль статистики

Формирование статистики осуществляется на основе данных, сохранённых в таблице games. Модуль содержит несколько функций, каждая из которых решает отдельную задачу.

Функция подсчёта распределения результатов выполняет три запроса к базе данных: определяет количество записей с результатом 1-0, количество записей с результатом 0-1 и количество записей с результатом 1/2-1/2. Эти значения используются для построения круговой и столбчатой диаграмм.

Функция определения популярных дебютов выполняет группировку записей по полю opening_name, подсчитывает количество вхождений каждого дебюта и сортирует результат по убыванию. На выходе возвращаются пять наиболее часто встречающихся дебютов.

Функция получения полного списка партий возвращает все записи из таблицы для отображения в виде таблицы на странице.

Процентное распределение результатов рассчитывается по следующей формуле:

$$P = (N_i / N) \times 100\%$$

где P — доля партий определённого результата в процентах;

N_i — количество партий с данным результатом;

N — общее количество загруженных партий.

Пользовательский интерфейс и визуализация

Интерфейс приложения реализован в виде одной HTML-страницы, которая формируется динамически при каждом запросе. Это означает, что содержимое страницы зависит от текущего состояния базы данных: если данные загружены, пользователь видит статистику, если нет — только форму загрузки.

На странице расположены следующие элементы. В верхней части находится заголовок приложения и краткое описание его назначения. Ниже размещена форма загрузки файла с кнопкой отправки. Под формой выводятся сообщения: зелёное — при успешной загрузке, красное — при ошибке.

После загрузки файла на странице появляются блоки со статистикой. Круговая диаграмма показывает соотношение побед белых, побед чёрных и ничьих. Рядом с ней располагается столбчатая диаграмма, позволяющая визуально сравнить количество побед каждой стороны. Обе диаграммы построены с помощью библиотеки Chart.js и формируются на основе данных, переданных из серверной части через шаблонизатор Jinja2.

Ниже диаграмм расположены две таблицы. Первая содержит пять наиболее популярных дебютов с указанием числа их встречаемости. Вторая

— полный список загруженных партий с указанием имён игроков, результата, даты, названия дебюта и количества ходов.

Внешний вид интерфейса до загрузки файла представлен на рисунке 9.

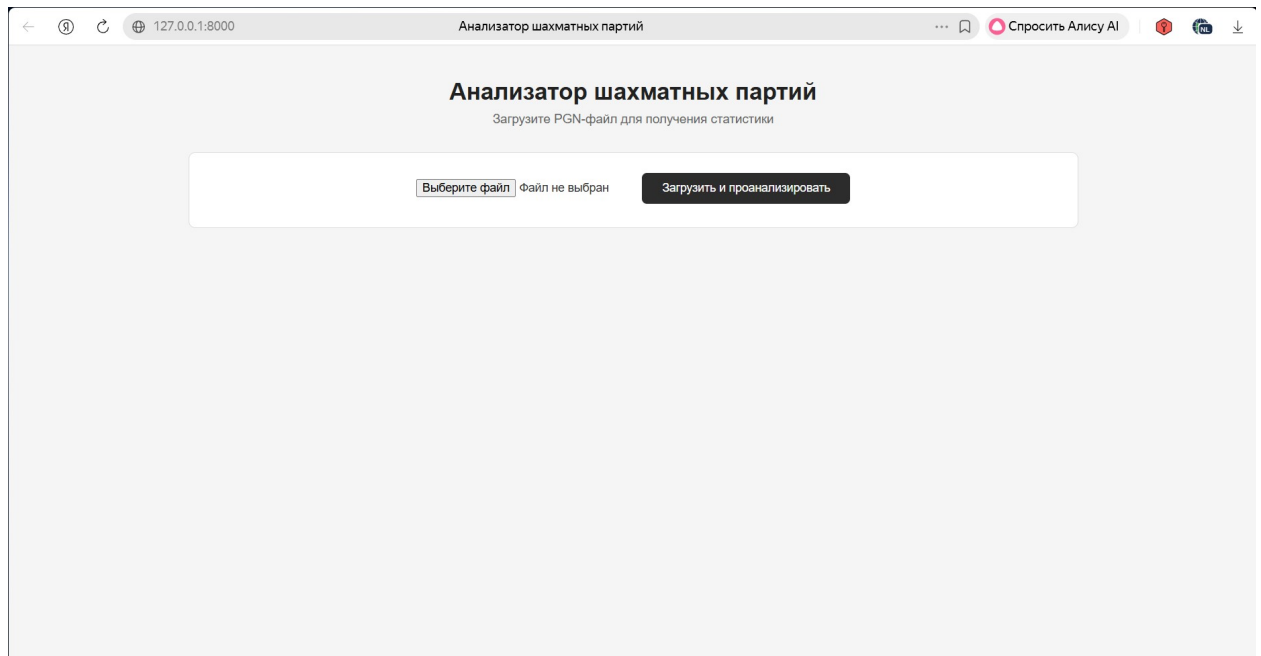


Рисунок 9 - Главная страница до загрузки файла

Результат успешной загрузки PGN-файла представлен на рисунке 10.

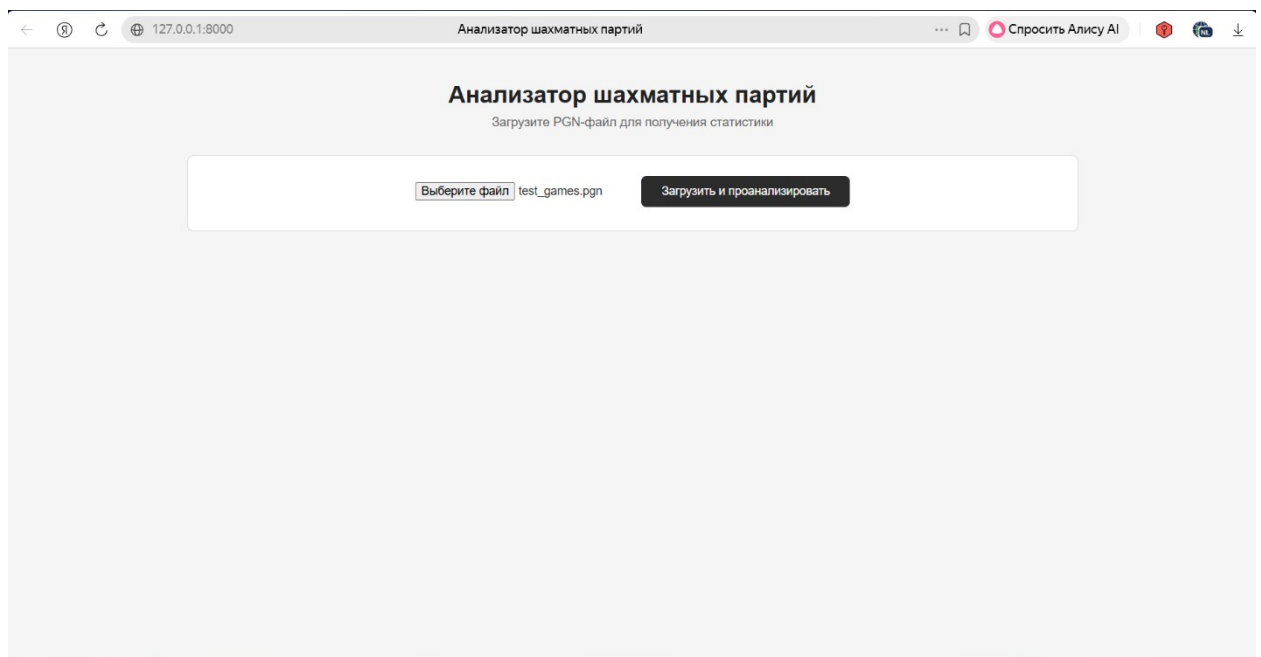


Рисунок 10 - Страница после загрузки файла

Блок диаграмм с распределением результатов партий показан на рисунке 11.

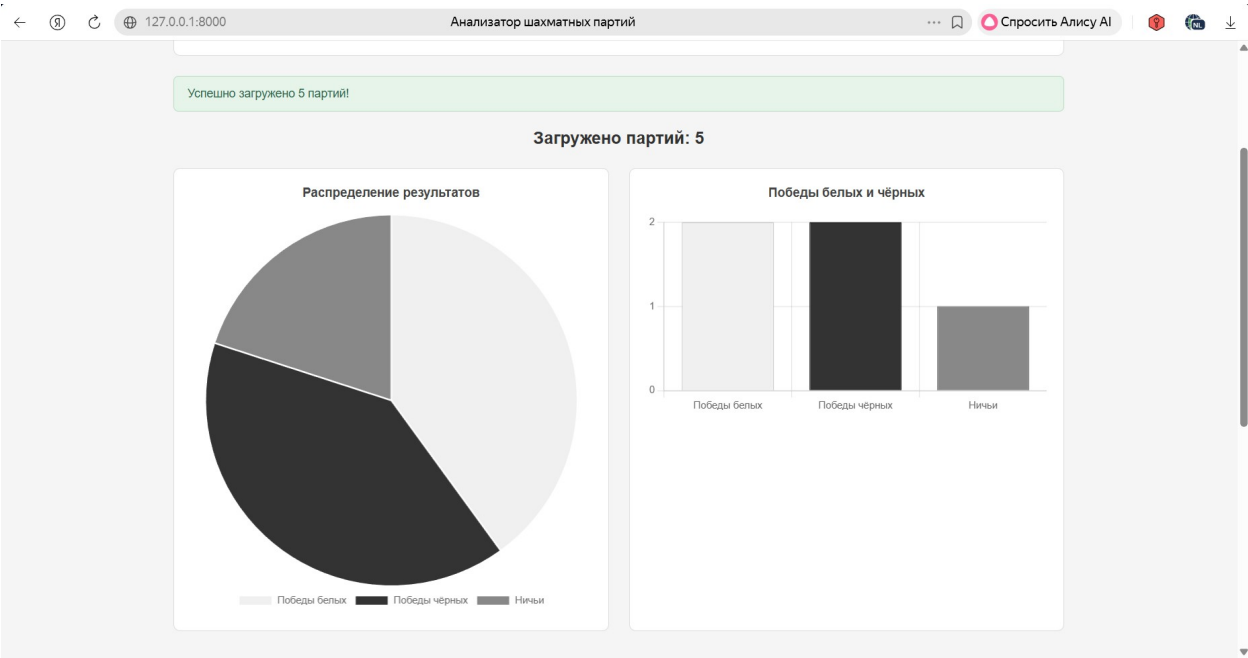


Рисунок 11 -Диаграммы распределения результатов

Таблица загруженных партий с указанием игроков, результата и дебюта представлена на рисунке 12.

Все загруженные партии						
№	Белые	Чёрные	Результат	Дата	Дебют	Ходов
1	Ivanov	Petrov	1-0	2026.04.14	Bishop's Opening	7
2	Sidorov	Smirnov	0-1	2026.04.14	Barnes Opening	4
3	Alexeev	Nikolaev	1/2-1/2	2026.04.14	Reti Opening	6
4	Volkov	Morozov	1-0	2026.04.14	Italian Game	15
5	Fedorov	Orlov	0-1	2026.04.14	Italian Game	14

Рисунок 12 - Таблица загруженных партий

Представленные скриншоты подтверждают, что разработанная информационная система выполняет все заявленные функции: принимает PGN-файл, извлекает из него данные, сохраняет их и представляет пользователю в виде наглядной статистики. Интерфейс прост, не перегружен лишними элементами и позволяет получить результат за минимальное число действий.

Ключевые фрагменты исходного кода приведены в Приложении В (см. Приложение В).

2.5. Тестирование и обеспечение качества

После завершения программной реализации было проведено функциональное тестирование информационной системы. Целью тестирования являлась проверка корректности работы каждого модуля и системы в целом, а также подтверждение соответствия результатов работы приложения требованиям, сформулированным в аналитической части.

Тестирование выполнялось вручную, на основе заранее подготовленных тестовых сценариев. Каждый сценарий описывал определённую ситуацию использования системы: от штатной загрузки корректного файла до попытки загрузить пустой документ или файл неверного формата. Такой подход позволяет проверить не только основной путь работы приложения, но и его поведение в нестандартных или ошибочных ситуациях.

Для проведения тестов были подготовлены несколько файлов:

- корректный PGN-файл, содержащий одну партию;
- корректный PGN-файл с пятью партиями;
- корректный PGN-файл с пятьюдесятью партиями;
- пустой файл с расширением .pgn;
- текстовый файл с расширением .txt;
- PGN-файл, в котором у части партий отсутствует тег Opening;
- PGN-файл, в котором у части партий отсутствует тег Date.

Результаты проведённого тестирования представлены в таблице 6.

Таблица 6.

Результаты функционального тестирования

№	Тестовый сценарий	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Загрузка файла с одной партией	valid_1.pgn	Партия загружается, статистика отображается	Соответствует	Пройден
2	Загрузка файла с пятью партиями	valid_5.pgn	Все партии обрабатываются, диаграммы корректны	Соответствует	Пройден
3	Загрузка файла с пятьюдесятью партиями	valid_50.pgn	Все партии обрабатываются за приемлемое время	Соответствует	Пройден
4	Загрузка пустого файла	empty.pgn	Сообщение об ошибке: файл пустой	Соответствует	Пройден
5	Загрузка файла неверного формата	test.txt	Сообщение об ошибке: неверный формат	Соответствует	Пройден
6	Загрузка PGN без тега Opening	no_opening.pgn	Партии сохраняются, поле дебюта пустое	Соответствует	Пройден
7	Загрузка PGN без тега Date	no_date.pgn	Партии сохраняются, поле даты пустое	Соответствует	Пройден
8	Проверка круговой диаграммы	valid_5.pgn	Значения на диаграмме совпадают с данными	Соответствует	Пройден
9	Проверка таблицы	valid_5.pgn	Отображаются наиболее частые	Соответствует	Пройден

	дебютов		дебюты		
10	Повторная загрузка нового файла	valid_1.pgn → valid_5.pgn	Старые данные удалены, новые загружены	Соответствует	Пройден

Анализ результатов тестирования показал, что все подготовленные сценарии были пройдены успешно. Система корректно обрабатывает файлы различного объёма, начиная от одной партии и заканчивая набором из нескольких десятков записей. При загрузке некорректных данных приложение не завершает работу аварийно, а выводит пользователю понятное сообщение об ошибке. Это подтверждает выполнение требования к надёжности, зафиксированного в разделе 1.5.

Отдельно была проверена работа системы с неполными PGN-записями. На практике файлы, загружаемые с различных платформ, могут содержать партии с разной степенью заполненности тегов. Тестирование показало, что отсутствие отдельных полей, таких как название дебюта или дата, не приводит к сбою. Партия сохраняется в базу данных, а пустые поля отображаются в таблице с соответствующей пометкой.

Была также проверена корректность визуализации. Значения, отображаемые на круговой и столбчатой диаграммах, совпали с фактическими данными, полученными из базы. Таблица популярных дебютов правильно отражала частоту встречаемости дебютных схем в загруженном наборе партий.

Помимо проверки основного функционала, было протестировано поведение системы при повторной загрузке файла. После загрузки нового файла старые данные полностью удаляются из базы, и статистика пересчитывается на основании нового набора партий. Это соответствует проектному решению, согласно которому система работает в рамках одного сеанса анализа.

Несмотря на успешное прохождение всех тестовых сценариев, следует отметить ряд ограничений, выявленных в ходе проверки.

Во-первых, при работе с файлами, содержащими более тысячи партий, время обработки может заметно возрасти. Для учебного проекта это не является критичным, однако в случае промышленного использования потребуется оптимизация алгоритма парсинга или применение асинхронной обработки.

Во-вторых, качество полученной статистики напрямую зависит от полноты заполнения тегов в исходном файле. Если значительная часть партий не содержит информации о дебюте, таблица популярных дебютов окажется малоинформативной.

В-третьих, система не предусматривает одновременную работу нескольких пользователей. Это ограничение обусловлено тем, что при каждой загрузке данные в базе полностью обновляются. Для многопользовательского режима потребовалась бы иная архитектура хранения данных.

Перечисленные ограничения являются допустимыми в рамках учебного проекта и не влияют на выполнение основных функциональных требований. Подробные материалы тестирования, включая тестовые файлы и дополнительные скриншоты, представлены в Приложении Д (см. Приложение Д).

2.6. Руководство пользователя

Настоящий раздел содержит описание порядка использования разработанной информационной системы. Руководство предназначено для конечного пользователя и охватывает основные действия: запуск приложения, загрузку файла, просмотр статистики и повторный анализ.

Назначение системы

Информационная система предназначена для статистического анализа шахматных партий, сохранённых в формате PGN. Пользователь загружает файл, содержащий одну или несколько партий, после чего система автоматически извлекает данные и отображает результаты в виде диаграмм и таблиц. Система не требует регистрации, авторизации или установки дополнительного программного обеспечения на стороне пользователя.

Системные требования

Для работы с приложением на стороне пользователя достаточно любого современного веб-браузера: Google Chrome, Mozilla Firefox, Microsoft Edge или аналогичного. Подключение к сети Интернет требуется только для загрузки библиотеки визуализации Chart.js при первом открытии страницы.

Для запуска серверной части на локальном компьютере необходимо наличие следующих компонентов:

- операционная система Windows, macOS или Linux;
- Python версии 3.8 или выше;
- установленные зависимости проекта, перечисленные в файле requirements.txt.

Запуск приложения

Запуск приложения выполняется в несколько шагов. Прежде всего необходимо открыть терминал или командную строку и перейти в каталог проекта. Затем следует активировать виртуальное окружение. В операционной системе Windows для этого используется команда:

```
venv\Scripts\activate
```

После активации окружения необходимо убедиться, что все зависимости установлены. Установка выполняется командой:

```
pip install -r requirements.txt
```

Для запуска сервера используется команда:

```
uvicorn main:app --reload
```

После выполнения этой команды в терминале появится сообщение о том, что сервер запущен и доступен по адресу <http://127.0.0.1:8000>. Для открытия приложения необходимо ввести этот адрес в адресной строке браузера.

Порядок работы с приложением

После открытия главной страницы пользователь видит форму загрузки файла. Для начала анализа необходимо выполнить следующие действия:

1. Нажать кнопку выбора файла и указать PGN-файл, хранящийся на компьютере.
2. Нажать кнопку «Загрузить и проанализировать».
3. Дождаться завершения обработки.

После успешной загрузки на странице отображается сообщение с указанием количества обработанных партий. Ниже появляются следующие элементы:

- круговая диаграмма, показывающая соотношение побед белых, побед чёрных и ничьих;
- столбчатая диаграмма, сравнивающая количество побед каждой стороны;
- таблица пяти наиболее популярных дебютов с указанием частоты их встречаемости;
- полная таблица всех загруженных партий с данными об игроках, результате, дате, дебюте и числе ходов.

Пользователь может свободно просматривать все представленные элементы, прокручивая страницу вниз.

Обработка ошибок

Если загруженный файл не соответствует требованиям, система выводит сообщение об ошибке. Возможны следующие ситуации:

- файл имеет расширение, отличное от .pgn — система сообщит о неверном формате;
- файл является пустым — система сообщит, что файл не содержит данных;
- файл не содержит корректных шахматных записей — система сообщит, что партии не найдены.

Во всех перечисленных случаях работа приложения не прерывается. Пользователю достаточно выбрать другой файл и повторить загрузку.

Повторная загрузка и очистка данных

Для анализа нового набора партий пользователь может просто выбрать другой файл и нажать кнопку загрузки. Предыдущие данные будут автоматически удалены из базы, и статистика пересчитается на основании нового файла.

Кроме того, на странице доступна кнопка «Очистить данные», которая позволяет вручную удалить все загруженные записи и вернуть приложение в исходное состояние.

Завершение работы

Для завершения работы с приложением достаточно закрыть вкладку браузера. Остановка серверной части выполняется в терминале нажатием сочетания клавиш Ctrl+C.

Скриншоты пользовательского интерфейса разработанного приложения приведены в приложении (см. Приложение Г).

Выводы по главе 2

Во второй главе были описаны результаты практической работы по проектированию и реализации информационной системы для статистического анализа шахматных партий формата PGN.

На этапе проектирования была определена клиент-серверная архитектура веб-приложения, выделены основные компоненты системы и описано их взаимодействие. Для формального представления логики работы были построены диаграммы в нотациях IDEF0 и UML: диаграмма вариантов использования, диаграмма активности и диаграмма последовательности. Это позволило зафиксировать структуру системы и порядок обработки данных до начала написания кода.

Была спроектирована база данных на основе СУБД SQLite. Определена структура таблицы games, описаны типы и ограничения полей. Для наглядного представления модели данных построены ER-диаграмма и диаграмма классов UML.

Выбор технологического стека был обоснован с учётом задач проекта и условий разработки. В качестве основных инструментов были выбраны Python, FastAPI, SQLite, python-chess, Jinja2 и Chart.js. Каждая технология была рассмотрена с точки зрения её соответствия поставленной задаче.

На этапе реализации были разработаны все запланированные модули: загрузка файлов, парсинг PGN, хранение данных, формирование статистики и визуализация результатов. Пользовательский интерфейс обеспечивает наглядное представление данных в виде круговой и столбчатой диаграмм, таблицы популярных дебютов и полного списка загруженных партий.

Функциональное тестирование подтвердило корректность работы системы. Все подготовленные тестовые сценарии были пройдены успешно. Система устойчиво обрабатывает как корректные, так и ошибочные входные

данные, выводя пользователю понятные сообщения при возникновении проблем.

Подготовлено руководство пользователя, описывающее порядок запуска приложения, загрузки файлов, просмотра результатов и обработки ошибок.

Таким образом, практическая часть работы завершена в полном объёме. Разработанная информационная система соответствует требованиям, сформулированным в аналитической части, и представляет собой работающий программный продукт, готовый к демонстрации и использованию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была решена задача разработки информационной системы для статистического анализа шахматных партий формата PGN.

На начальном этапе была изучена предметная область, связанная с цифровым хранением и обработкой шахматных партий. Было установлено, что формат PGN является наиболее распространённым стандартом записи шахматных игр и обладает удобной для программной обработки текстовой структурой. Вместе с тем было выявлено, что существующие решения для работы с шахматными данными либо ориентированы на профессиональных пользователей и обладают избыточным функционалом, либо сосредоточены на процессе игры, а не на статистическом анализе пользовательских архивов.

На основании проведённого анализа была сформулирована задача создания простого и доступного веб-приложения, позволяющего загружать PGN-файлы, автоматически извлекать из них метаданные о партиях и представлять результаты в наглядной графической форме. Были определены функциональные и нефункциональные требования к системе, выполнен сравнительный анализ аналогов и проведён SWOT-анализ разрабатываемого решения.

В практической части была спроектирована архитектура приложения, построены диаграммы функционального моделирования и UML-диаграммы, описана структура базы данных. Для реализации был выбран стек технологий, включающий Python, FastAPI, SQLite, python-chess, Jinja2 и Chart.js. Выбор каждого инструмента был обоснован с точки зрения его соответствия задачам проекта.

В результате реализации была создана информационная система, обеспечивающая следующие возможности:

- загрузка PGN-файлов через веб-интерфейс;
- автоматический разбор одной или нескольких партий из файла;
- извлечение метаданных: имён игроков, результата, даты, кода и названия дебюта, количества ходов;
- сохранение данных в реляционной базе данных;
- отображение круговой диаграммы распределения результатов;
- отображение столбчатой диаграммы побед белых и чёрных;
- формирование таблицы наиболее популярных дебютов;
- вывод полного списка загруженных партий.

Проведённое функциональное тестирование подтвердило работоспособность системы и её соответствие предъявленным требованиям.

Приложение корректно обрабатывает как валидные, так и ошибочные входные данные, обеспечивая устойчивость к неполным записям и некорректным файлам.

Все задачи, поставленные во введении, были выполнены:

1. Проведён анализ предметной области, изучены существующие аналоги и сформулированы требования к системе.
2. Спроектирована архитектура веб-приложения, структура базы данных и выбран технологический стек.
3. Реализована программная система, проведено её тестирование и подготовлена документация.

Таким образом, цель выпускной квалификационной работы достигнута.

В качестве перспективных направлений развития проекта можно рассмотреть несколько возможных улучшений. Во-первых, добавление фильтрации партий по имени игрока, результату или дебюту позволило бы пользователю получать более детальную статистику. Во-вторых, реализация хранения истории загрузок дала бы возможность сравнивать результаты анализа разных файлов. В-третьих, интеграция с шахматным движком открыла бы путь к оценке качества ходов и более глубокому анализу партий. Наконец, добавление системы авторизации и многопользовательского режима позволило бы использовать приложение в качестве полноценного онлайн-сервиса.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Нормативные документы

1. Федеральный государственный образовательный стандарт среднего профессионального образования по специальности 09.02.07 «Информационные системы и программирование» : утв. приказом Минобрнауки России от 09.12.2016 № 1547.
2. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления. — М. : Стандартинформ, 2017.
3. ГОСТ 34.602–2020. Техническое задание на создание автоматизированной системы. — М. : Стандартинформ, 2020.
4. ГОСТ 19.201–78. Техническое задание. Требования к содержанию и оформлению. — М. : Издательство стандартов, 1978.
5. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. — М. : Стандартинформ, 2018.
6. ГОСТ 19.105–78. Общие требования к программным документам. — М. : Издательство стандартов, 1978.
7. ГОСТ 19.401–78. Текст программы. Требования к содержанию и оформлению. — М. : Издательство стандартов, 1978.
8. ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
9. ISO/IEC 12207:2017. Systems and software engineering — Software life cycle processes.

Основная литература

10. Лутц М. Изучаем Python / М. Лутц. — 5-е изд. — СПб. : Вильямс, 2020. — 1216 с.

11. Свейгарт Э. Автоматизация рутинных задач с помощью Python / Э. Свейгарт. — М. : Вильямс, 2021. — 592 с.
12. Гринберг М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг. — 2-е изд. — М. : ДМК Пресс, 2020. — 512 с.
13. Федорова Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие / Г. Н. Федорова. — М. : КУРС : ИНФРА-М, 2021. — 336 с.
14. Фуфаев Э. В. Базы данных : учебное пособие / Э. В. Фуфаев, Д. Э. Фуфаев. — 10-е изд. — М. : Академия, 2020. — 320 с.
15. Стасышин В. М. Проектирование информационных систем и баз данных : учебное пособие / В. М. Стасышин. — Новосибирск : НГТУ, 2020. — 100 с.
16. Коннолли Т. Базы данных: проектирование, реализация и управление / Т. Коннолли, К. Бегг. — 3-е изд. — М. : Вильямс, 2021. — 1440 с.
17. Фаулер М. UML. Основы / М. Фаулер. — 3-е изд. — СПб. : Символ-Плюс, 2019. — 192 с.
18. Буч Г. Объектно-ориентированный анализ и проектирование с примерами приложений / Г. Буч, Р. А. Максимчук, М. У. Энгл. — 3-е изд. — М. : Вильямс, 2020. — 720 с.
19. Соммервилл И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М. : Вильямс, 2020. — 960 с.
20. Хорстман К. Python для нетерпеливых / К. Хорстман. — М. : ДМК Пресс, 2021. — 352 с.
21. Маркин А. В. Основы веб-программирования : учебное пособие / А. В. Маркин. — М. : ИНФРА-М, 2021. — 285 с.

22. Бизли Д. Python. Справочник. Полное описание языка / Д. Бизли. — М. : Вильямс, 2020. — 720 с.

Дополнительная литература

23. Элбон К. Машинное обучение с использованием Python. Сборник рецептов / К. Элбон. — СПб. : БХВ-Петербург, 2020. — 384 с.
24. Рамальо Л. Python. К вершинам мастерства / Л. Рамальо. — 2-е изд. — М. : ДМК Пресс, 2022. — 896 с.
25. Мюллер Дж. П. Искусственный интеллект для чайников / Дж. П. Мюллер, Л. Массарон. — М. : Вильямс, 2020. — 384 с.
26. Мэтиз Э. Изучаем Python. Программирование игр, визуализация данных, веб-приложения / Э. Мэтиз. — 3-е изд. — СПб. : Питер, 2023. — 512 с.
27. Дронов В. А. Django 3.0. Практика создания веб-сайтов на Python / В. А. Дронов. — СПб. : БХВ-Петербург, 2021. — 704 с.

Электронные ресурсы

28. Portable Game Notation Specification and Implementation Guide [Электронный ресурс]. — Режим доступа: <https://www.saremba.de/chessgml/standards/pgn/pgn-complete.htm>.
29. Python Software Foundation. Python Documentation [Электронный ресурс]. — Режим доступа: <https://docs.python.org/>.
30. FastAPI Documentation [Электронный ресурс]. — Режим доступа: <https://fastapi.tiangolo.com/>.
31. SQLite Documentation [Электронный ресурс]. — Режим доступа: <https://www.sqlite.org/docs.html>.

32. Chart.js Documentation [Электронный ресурс]. — Режим доступа: <https://www.chartjs.org/docs/latest/>.
33. python-chess Documentation [Электронный ресурс]. — Режим доступа: <https://python-chess.readthedocs.io/>.
34. Lichess.org — бесплатная шахматная платформа [Электронный ресурс]. — Режим доступа: <https://lichess.org/>.
35. Chess.com — шахматная платформа [Электронный ресурс]. — Режим доступа: <https://www.chess.com/>.
36. SCID — Shane's Chess Information Database [Электронный ресурс]. — Режим доступа: <https://scid.sourceforge.net/>.
37. ChessBase — профессиональная шахматная база данных [Электронный ресурс]. — Режим доступа: <https://www.chessbase.com/>.
38. SQLAlchemy Documentation [Электронный ресурс]. — Режим доступа: <https://www.sqlalchemy.org/>.
39. Jinja2 Documentation [Электронный ресурс]. — Режим доступа: <https://jinja.palletsprojects.com/>.
40. MDN Web Docs. HTML, CSS, JavaScript [Электронный ресурс]. — Режим доступа: <https://developer.mozilla.org/>.

ПРИЛОЖЕНИЯ

Приложение А

Техническое задание на разработку информационной системы для статистического анализа шахматных партий формата PGN.

1. Общие сведения

- 1.1. Полное наименование системы: «Информационная система для статистического анализа шахматных партий формата PGN».
- 1.2. Краткое наименование: «Анализатор шахматных партий».
- 1.3. Заказчик: ГБПОУ «Колледж малого бизнеса № 4».
- 1.4. Разработчик: студент группы ИПО-43 Гаврилин Александр Анатольевич.
- 1.5. Основание для разработки: задание на выпускную квалификационную работу по специальности 09.02.07 «Информационные системы и программирование».

2. Назначение и цели создания системы

- 2.1. Назначение системы: автоматизация процесса статистической обработки шахматных партий, сохранённых в формате PGN, и представление результатов анализа в наглядной графической форме.
- 2.2. Цель создания системы: разработка веб-приложения, позволяющего пользователю загрузить PGN-файл, автоматически извлечь из него метаданные о партиях, сохранить данные и получить статистику в виде диаграмм и таблиц.
- 2.3. Целевая аудитория: шахматисты-любители, которым необходим простой инструмент для анализа собственных партий без использования сложного профессионального программного обеспечения.

3. Требования к системе

3.1. Функциональные требования

3.1.1. Модуль загрузки данных.

Система должна предоставлять веб-форму для загрузки файла с расширением .pgn. Должна выполняться проверка, что файл не является пустым и имеет допустимое расширение. Система должна поддерживать загрузку файла, содержащего множество партий.

3.1.2. Модуль парсинга.

Система должна автоматически извлекать из каждой партии следующие данные:

- имя игрока белыми (тег White);
- имя игрока чёрными (тег Black);
- результат партии (тег Result);
- дату проведения (тег Date);
- код дебюта (тег ECO);
- название дебюта (тег Opening);
- количество ходов.

При отсутствии отдельных тегов система должна подставлять значение по умолчанию и продолжать обработку.

3.1.3. Модуль хранения данных.

Извлечённые данные должны сохраняться в реляционной базе данных. Перед записью нового набора данных предыдущие записи должны удаляться.

3.1.4. Модуль статистики и визуализации.

Система должна рассчитывать и отображать:

- круговую диаграмму распределения результатов (победы белых, победы чёрных, ничьи);

- столбчатую диаграмму сравнения побед белых и чёрных;
- таблицу пяти наиболее популярных дебютов;
- полную таблицу всех загруженных партий.

3.1.5. Модуль управления данными.

Система должна предоставлять возможность ручной очистки загруженных данных.

3.2. Нефункциональные требования

3.2.1. Удобство использования: интерфейс должен быть понятным, количество действий для получения результата — не более трёх.

3.2.2. Надёжность: система должна устойчиво работать при загрузке файлов с неполными данными и при попытке загрузить файл неверного формата. Во всех случаях должно выводиться понятное сообщение об ошибке.

3.2.3. Производительность: обработка файла, содержащего до 100 партий, должна выполняться не более чем за 5 секунд.

3.2.4. Кроссплатформенность: приложение должно корректно работать в любом современном веб-браузере.

3.2.5. Сопровождаемость: код должен быть организован по модульному принципу с разделением логики по файлам.

4. Требования к составу и содержанию работ

Разработка системы включает следующие этапы:

4.1. Анализ предметной области и изучение формата PGN.

4.2. Анализ существующих аналогов.

4.3. Формулирование требований к системе.

4.4. Проектирование архитектуры веб-приложения.

- 4.5. Проектирование структуры базы данных.
- 4.6. Выбор и обоснование технологического стека.
- 4.7. Реализация модулей загрузки, парсинга, хранения, статистики и визуализации.
- 4.8. Разработка пользовательского интерфейса.
- 4.9. Тестирование системы.
- 4.10. Подготовка документации и руководства пользователя.

5. Требования к программному и техническому обеспечению

5.1. Серверная часть (для запуска приложения):

- операционная система: Windows 10 и выше, macOS или Linux;
- Python версии 3.8 и выше;
- установленные библиотеки: FastAPI, uvicorn, SQLAlchemy, python-chess, Jinja2, python-multipart;
- СУБД: SQLite (встроенная, установка не требуется).

5.2. Клиентская часть (для использования приложения):

- любой современный веб-браузер (Google Chrome, Mozilla Firefox, Microsoft Edge);
- подключение к сети Интернет для загрузки библиотеки Chart.js при первом открытии страницы.

6. Требования к входным и выходным данным

6.1. Входные данные:

- текстовый файл формата PGN (.pgn);
- файл может содержать одну или несколько шахматных партий;

- каждая партия может включать теги: Event, Site, Date, White, Black, Result, ECO, Opening и последовательность ходов.

6.2. Выходные данные:

- количество загруженных партий;
- распределение результатов: число побед белых, побед чёрных и ничьих;
- круговая диаграмма распределения результатов;
- столбчатая диаграмма побед белых и чёрных;
- таблица пяти наиболее популярных дебютов;
- полная таблица загруженных партий с указанием игроков, результата, даты, дебюта и числа ходов.

7. Порядок приёмки

7.1. Приёмка системы осуществляется путём демонстрации работоспособности приложения на заседании Государственной экзаменационной комиссии.

7.2. Критерии приёмки:

- система запускается и открывается в браузере;
- загрузка корректного PGN-файла приводит к отображению статистики;
- диаграммы и таблицы формируются корректно;
- при загрузке некорректного файла выводится сообщение об ошибке;
- очистка данных работает без сбоев.

8. Сроки выполнения

8.1. Дата выдачи задания: 14 апреля 2026 года.

8.2. Дата сдачи готовой работы: 17 мая 2026 года.

Приложение Б

Диаграммы и модели системы.

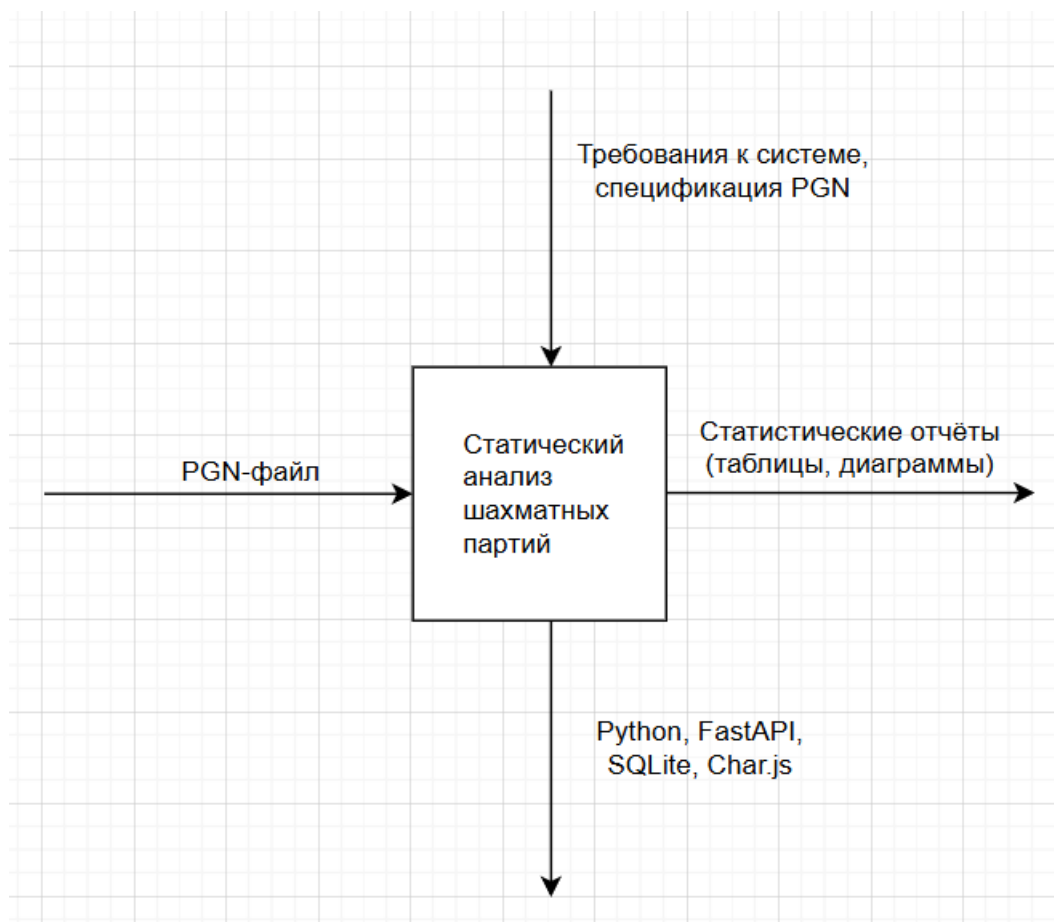


Рисунок Б.1 - Контекстная диаграмма IDEF0 (уровень A0)

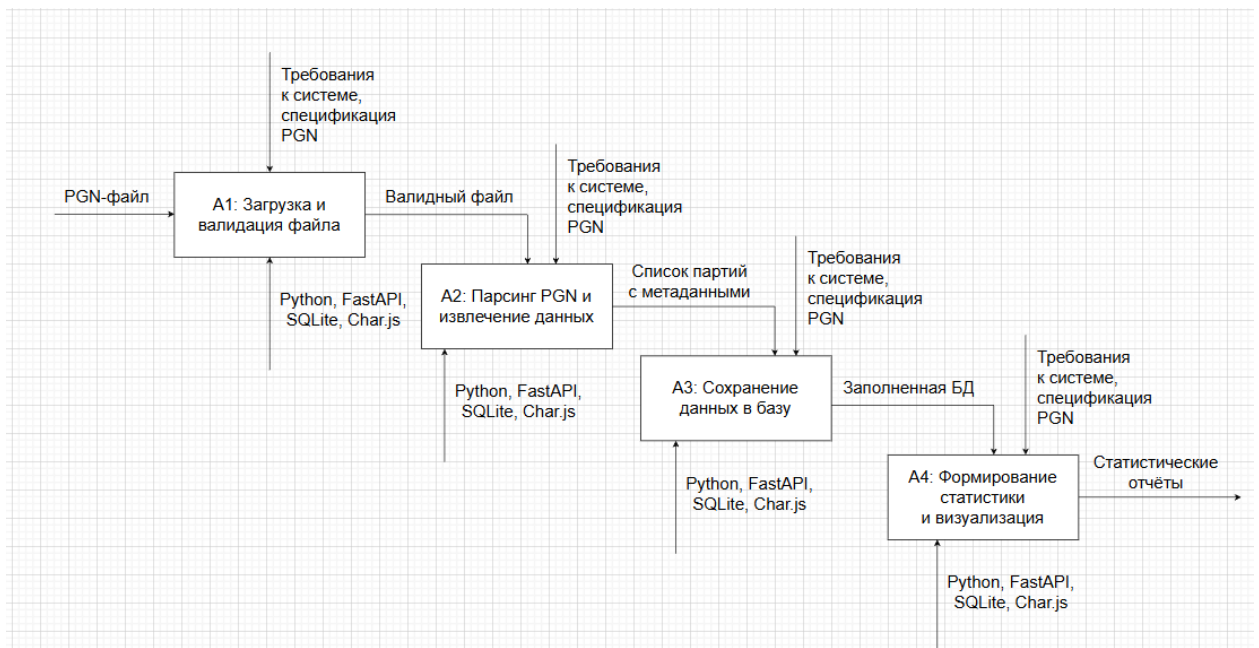


Рисунок Б.2 — Диаграмма декомпозиции IDEF0 (уровень A1)

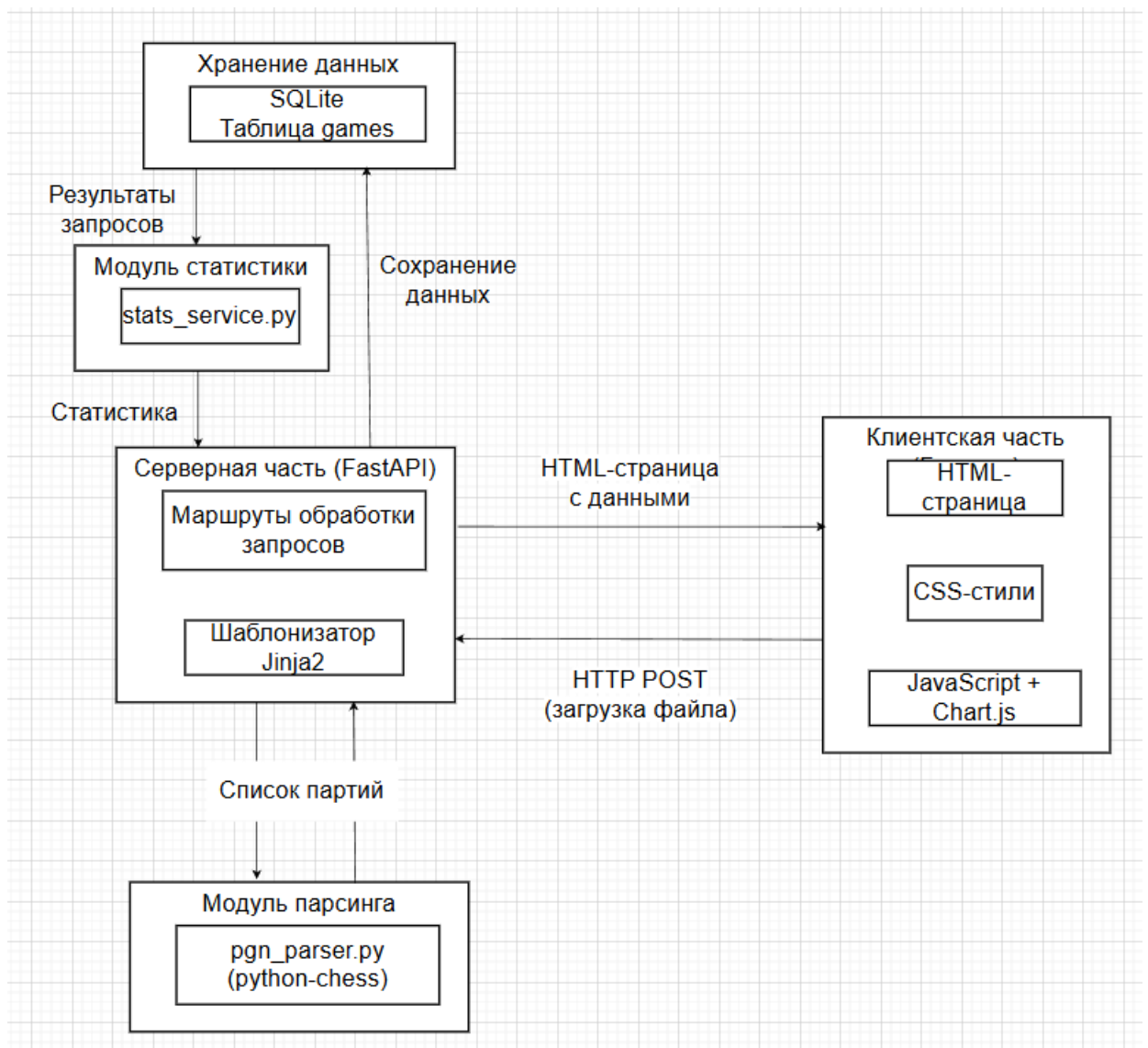


Рисунок Б.3 - Общая архитектура информационной системы

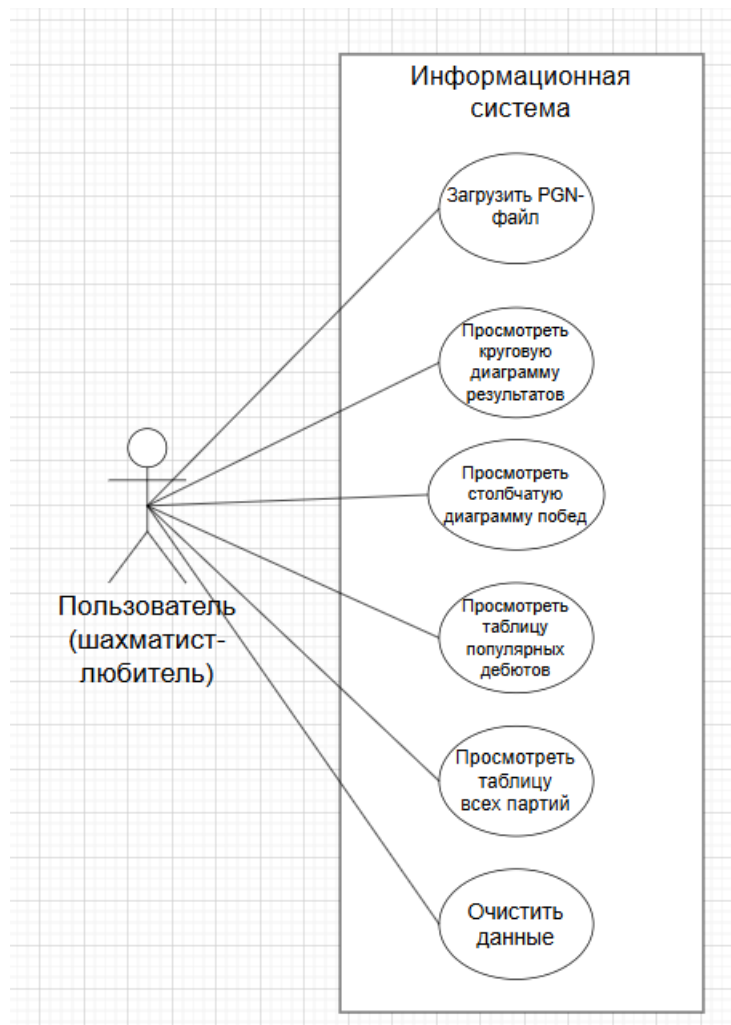


Рисунок Б.4 - Диаграмма вариантов использования (UML Use Case)

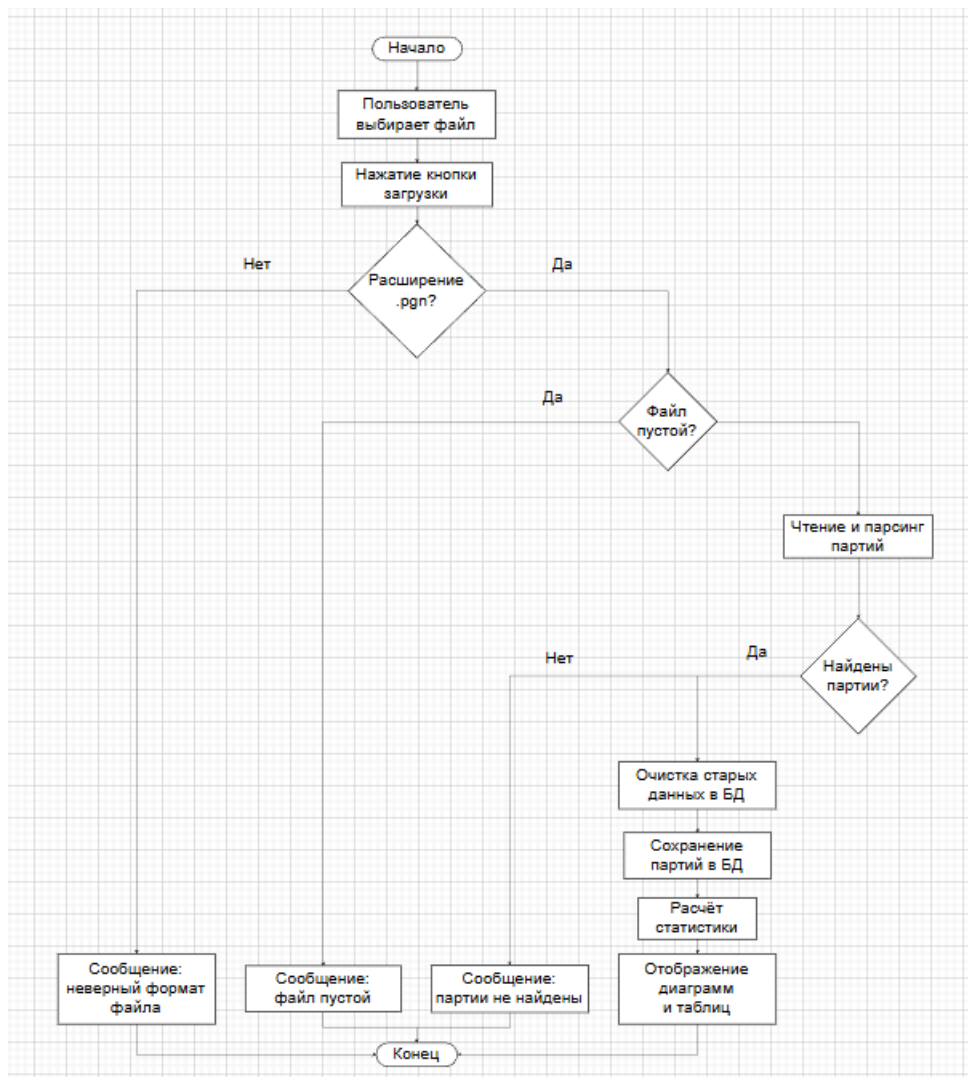


Рисунок Б.5 - Диаграмма активности процесса обработки PGN-файла (UML Activity)

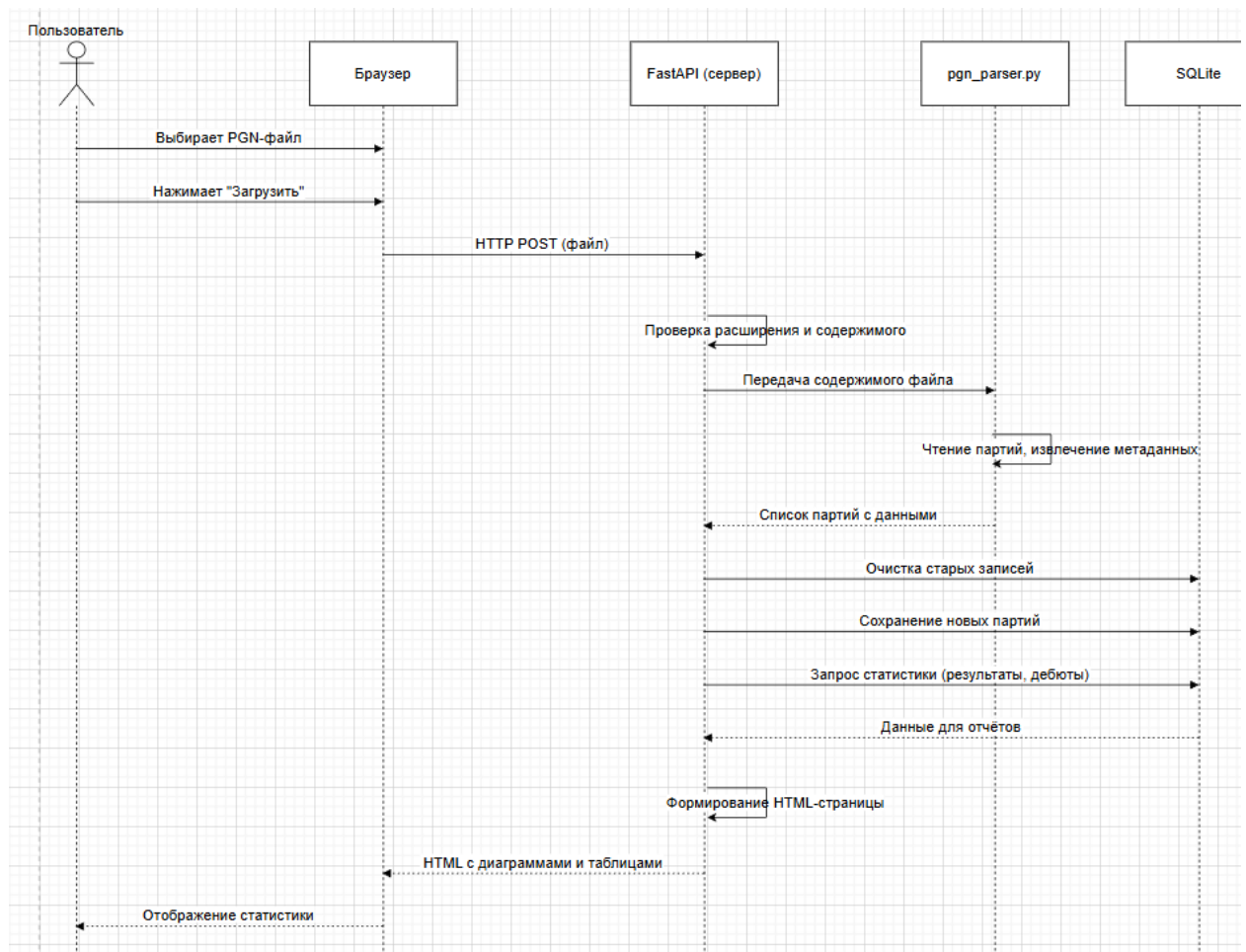


Рисунок Б.6 — Диаграмма последовательности взаимодействия компонентов (UML Sequence)

Games			
INTEGER	id	PK	Уникальный идентификатор
TEXT	white_player		Имя игрока белыми
TEXT	black_player		Имя игрока чёрными
TEXT	result		Результат партии
TEXT	game_date		Дата партии
TEXT	eco_code		Код дебюта ECO
TEXT	opening_name		Название дебюта
INTEGER	move_count		Количество ходов

Рисунок Б.7 — ER-диаграмма базы данных

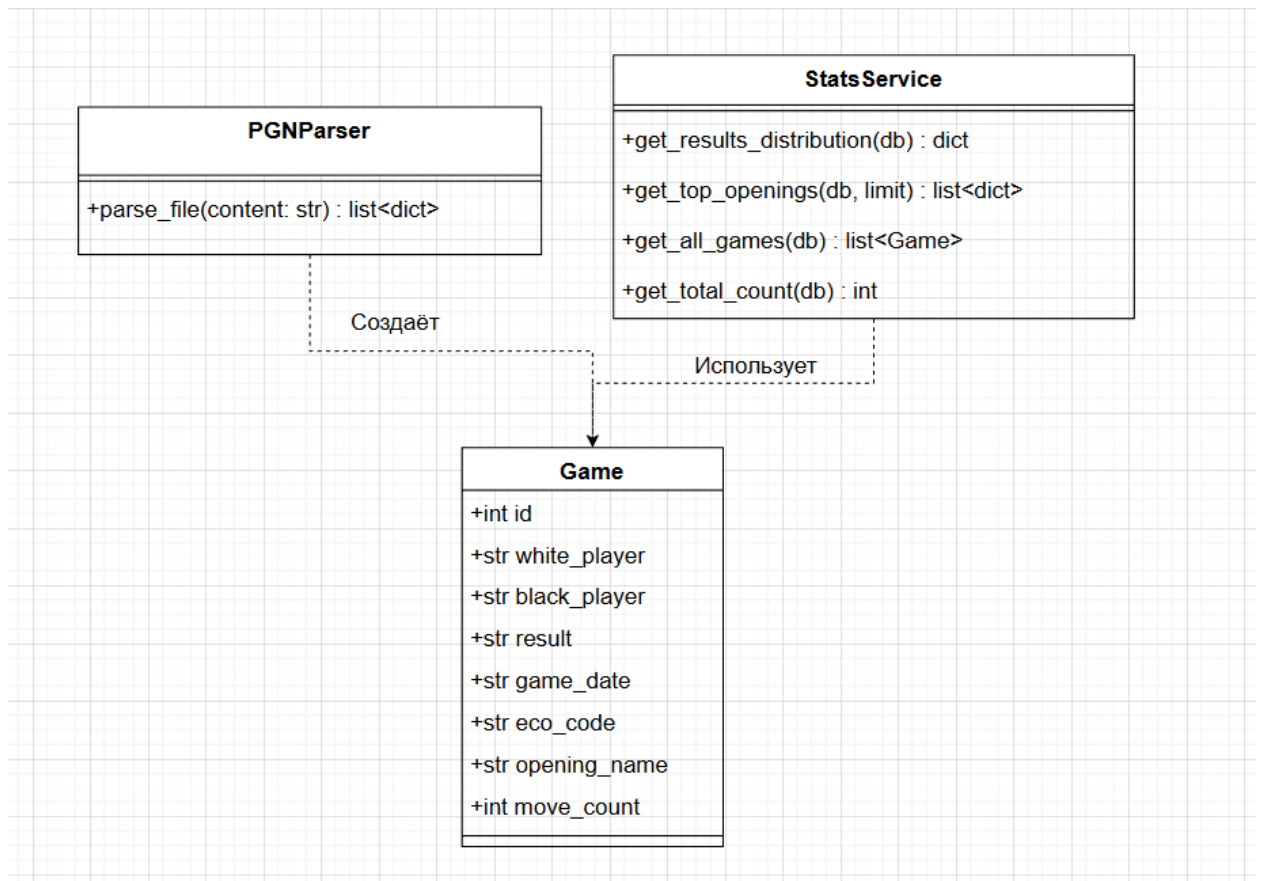


Рисунок Б.8 — Диаграмма классов (UML Class Diagram)

Приложение В

Листинги программного кода.

Листинг В.1 — Модуль подключения к базе данных (database.py)

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# Путь к файлу базы данных SQLite
SQLALCHEMY_DATABASE_URL = "sqlite:///./chess_games.db"

# Создаём движок базы данных
engine = create_engine(
    SQLALCHEMY_DATABASE_URL,
    connect_args={"check_same_thread": False}
)

# Создаём фабрику сессий
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
                             bind=engine)

# Базовый класс для моделей
Base = declarative_base()

def get_db():
    """Функция для получения сессии базы данных."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Листинг В.2 — Модель данных (models.py)

```
from sqlalchemy import Column, Integer, String
from database import Base
```

```

class Game(Base):
    """Модель таблицы games в базе данных."""
    __tablename__ = "games"

    id = Column(Integer, primary_key=True, index=True,
autoincrement=True)

    white_player = Column(String, nullable=False, default="Не указано")
    black_player = Column(String, nullable=False, default="Не указано")
    result = Column(String, nullable=False, default="Не указано")
    game_date = Column(String, nullable=True)
    eco_code = Column(String, nullable=True)
    opening_name = Column(String, nullable=True)
    move_count = Column(Integer, nullable=True)

```

Листинг В.3 — Модуль парсинга PGN-файлов (pgn_parser.py)

```

import chess.pgn
import io

def parse_pgn_file(file_content: str) -> list[dict]:
    """
    Принимает текстовое содержимое PGN-файла.
    Возвращает список словарей с данными о партиях.
    """
    games = []

    # Создаём текстовый поток из содержимого файла
    pgn_io = io.StringIO(file_content)

    while True:
        # Читаем очередную партию
        game = chess.pgn.read_game(pgn_io)

        # Если партий больше нет — выходим из цикла
        if game is None:
            break

```

```

# Извлекаем заголовки партии
headers = game.headers

# Подсчитываем количество ходов
move_count = 0
board = game.board()
for move in game.mainline_moves():
    board.push(move)
    move_count += 1

# Формируем словарь с данными партии
game_data = {
    "white_player": headers.get("White", "Не указано"),
    "black_player": headers.get("Black", "Не указано"),
    "result": headers.get("Result", "Не указано"),
    "game_date": headers.get("Date", None),
    "eco_code": headers.get("ECO", None),
    "opening_name": headers.get("Opening", None),
    "move_count": move_count if move_count > 0 else None,
}

games.append(game_data)

return games

```

Листинг В.4 — Модуль формирования статистики (stats_service.py)

```

from sqlalchemy.orm import Session
from sqlalchemy import func
from models import Game

def get_results_distribution(db: Session) -> dict:
    """
    Возвращает количество побед белых, побед чёрных и ничьих.
    """
    white_wins = db.query(Game).filter(Game.result == "1-0").count()

```

```

black_wins = db.query(Game).filter(Game.result == "0-1").count()
draws = db.query(Game).filter(Game.result == "1/2-1/2").count()

return {
    "white_wins": white_wins,
    "black_wins": black_wins,
    "draws": draws,
}

def get_top_openings(db: Session, limit: int = 5) -> list[dict]:
    """
    Возвращает топ-N самых популярных дебютов.
    """
    results = (
        db.query(Game.opening_name,
func.count(Game.opening_name).label("count"))
        .filter(Game.opening_name.isnot(None))
        .group_by(Game.opening_name)
        .order_by(func.count(Game.opening_name).desc())
        .limit(limit)
        .all()
    )

    return [{"opening": row.opening_name, "count": row.count} for row in
results]

def get_all_games(db: Session) -> list[Game]:
    """
    Возвращает список всех загруженных партий.
    """
    return db.query(Game).all()

def get_total_count(db: Session) -> int:
    """
    Возвращает общее количество партий в базе.
    """

```

```
return db.query(Game).count()
```

Листинг В.5 — Основной файл приложения (main.py)

```
from fastapi import FastAPI, File, UploadFile, Request, Depends
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from sqlalchemy.orm import Session

import database
import models
from pgn_parser import parse_pgn_file
from stats_service import (
    get_results_distribution,
    get_top_openings,
    get_all_games,
    get_total_count,
)

# Создаём приложение
app = FastAPI()

# Подключаем папку со статическими файлами (CSS)
app.mount("/static", StaticFiles(directory="static"), name="static")

# Подключаем папку с HTML-шаблонами
templates = Jinja2Templates(directory="templates")

# Создаём таблицы в базе данных при запуске
models.Base.metadata.create_all(bind=database.engine)

@app.get("/", response_class=HTMLResponse)
async def index(request: Request, db: Session = Depends(database.get_db)):
    """Главная страница — показывает статистику."""
```

```

total = get_total_count(db)

if total > 0:
    distribution = get_results_distribution(db)
    top_openings = get_top_openings(db)
    all_games = get_all_games(db)
else:
    distribution = {"white_wins": 0, "black_wins": 0, "draws": 0}
    top_openings = []
    all_games = []

return templates.TemplateResponse(
    request=request,
    name="index.html",
    context={
        "total": total,
        "distribution": distribution,
        "top_openings": top_openings,
        "all_games": all_games,
        "message": None,
        "error": None,
    }
)

@app.post("/upload", response_class=HTMLResponse)
async def upload_pgn(
    request: Request,
    file: UploadFile = File(...),
    db: Session = Depends(database.get_db),
):
    """Обработка загрузки PGN-файла."""

    error = None
    message = None

```

```

# Проверка расширения файла
if not file.filename.endswith(".pgn"):
    error = "Ошибка: файл должен иметь расширение .pgn"
else:
    # Читаем содержимое файла
    content_bytes = await file.read()

    # Проверка на пустой файл
    if len(content_bytes) == 0:
        error = "Ошибка: файл пустой"
    else:
        try:
            # Декодируем содержимое
            content_str = content_bytes.decode("utf-8")

            # Парсим партии
            games_data = parse_pgn_file(content_str)

            if len(games_data) == 0:
                error = "Ошибка: файл не содержит шахматных партий"
            else:
                # Очищаем старые данные перед новой загрузкой
                db.query(models.Game).delete()
                db.commit()

                # Сохраняем новые партии в базу
                for game_data in games_data:
                    game = models.Game(**game_data)
                    db.add(game)
                db.commit()

                message = f"Успешно загружено {len(games_data)}
партий!"

        except Exception as e:
            error = f"Ошибка при обработке файла: {str(e)}"

```

```

# Получаем статистику для отображения
total = get_total_count(db)

if total > 0:
    distribution = get_results_distribution(db)
    top_openings = get_top_openings(db)
    all_games = get_all_games(db)
else:
    distribution = {"white_wins": 0, "black_wins": 0, "draws": 0}
    top_openings = []
    all_games = []

return templates.TemplateResponse(
    request=request,
    name="index.html",
    context={
        "total": total,
        "distribution": distribution,
        "top_openings": top_openings,
        "all_games": all_games,
        "message": message,
        "error": error,
    }
)

@app.post("/clear", response_class=HTMLResponse)
async def clear_data(
    request: Request,
    db: Session = Depends(database.get_db),
):
    """Очистка всех данных из базы."""

    db.query(models.Game).delete()
    db.commit()

```

```

return templates.TemplateResponse(
    request=request,
    name="index.html",
    context={
        "total": 0,
        "distribution": {"white_wins": 0, "black_wins": 0, "draws":
0},
        "top_openings": [],
        "all_games": [],
        "message": "Данные успешно очищены.",
        "error": None,
    }
)

```

Листинг В.6 — HTML-шаблон главной страницы (index.html)

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Анализатор шахматных партий</title>
    <link rel="stylesheet" href="/static/style.css">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>

    <div class="container">

        <h1>Анализатор шахматных партий</h1>
        <p class="subtitle">Загрузите PGN-файл для получения
статистики</p>

        <!-- Форма загрузки файла -->
        <div class="upload-section">
            <form action="/upload" method="post" enctype="multipart/form-
data">

```

```

        <input type="file" name="file" accept=".pgn" required>
        <button type="submit" class="btn-primary">
            Загрузить и проанализировать
        </button>
    </form>

    {% if total > 0 %}
        <form action="/clear" method="post" style="margin-top:
10px;">
            <button type="submit" class="btn-secondary">
                Очистить данные
            </button>
        </form>
    {% endif %}
</div>

<!-- Сообщения -->
{% if message %}
<div class="message success">{{ message }}</div>
{% endif %}

{% if error %}
<div class="message error">{{ error }}</div>
{% endif %}

<!-- Статистика (показываем только если есть данные) -->
{% if total > 0 %}

<div class="stats-header">
    <h2>Загружено партий: {{ total }}</h2>
</div>

<!-- Диаграммы -->
<div class="charts-section">

    <div class="chart-box">

```

```

        <h3>Распределение результатов</h3>
        <canvas id="pieChart"></canvas>
    </div>

    <div class="chart-box">
        <h3>Победы белых и чёрных</h3>
        <canvas id="barChart"></canvas>
    </div>

</div>

<!-- Топ дебютов -->
{% if top_openings %}
<div class="table-section">
    <h3>Топ-5 популярных дебютов</h3>
    <table>
        <thead>
            <tr>
                <th>№</th>
                <th>Дебют</th>
                <th>Количество партий</th>
            </tr>
        </thead>
        <tbody>
            {% for opening in top_openings %}
            <tr>
                <td>{{ loop.index }}</td>
                <td>{{ opening.opening }}</td>
                <td>{{ opening.count }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>
{% endif %}

```

```

<!-- Таблица всех партий -->
<div class="table-section">
    <h3>Все загруженные партии</h3>
    <table>
        <thead>
            <tr>
                <th>№</th>
                <th>Белые</th>
                <th>Чёрные</th>
                <th>Результат</th>
                <th>Дата</th>
                <th>Дебют</th>
                <th>Ходов</th>
            </tr>
        </thead>
        <tbody>
            {% for game in all_games %}
            <tr>
                <td>{{ loop.index }}</td>
                <td>{{ game.white_player }}</td>
                <td>{{ game.black_player }}</td>
                <td>{{ game.result }}</td>
                <td>{{ game.game_date or "-" }}</td>
                <td>{{ game.opening_name or "-" }}</td>
                <td>{{ game.move_count or "-" }}</td>
            </tr>
            {% endfor %}
        </tbody>
    </table>
</div>

{% endif %}

</div>

<!-- Скрипт диаграмм -->

```

```

        {% if total > 0 %}
<div
    id="chart-data"
    data-white-wins="{{ distribution.white_wins }}"
    data-black-wins="{{ distribution.black_wins }}"
    data-draws="{{ distribution.draws }}"
    style="display: none;"
></div>

<script>
    const chartDataElement = document.getElementById('chart-data');

    const whiteWins = Number(chartDataElement.dataset.whiteWins);
    const blackWins = Number(chartDataElement.dataset.blackWins);
    const draws = Number(chartDataElement.dataset.draws);

    // Круговая диаграмма
    const pieCtx =
document.getElementById('pieChart').getContext('2d');
    new Chart(pieCtx, {
        type: 'pie',
        data: {
            labels: ['Победы белых', 'Победы чёрных', 'Ничьи'],
            datasets: [{
                data: [whiteWins, blackWins, draws],
                backgroundColor: [
                    '#f0f0f0',
                    '#333333',
                    '#888888'
                ],
                borderColor: '#ffffff',
                borderWidth: 2
            }]
        },
        options: {
            responsive: true,

```

```

        plugins: {
            legend: {
                position: 'bottom'
            }
        }
    }
});

// Столбчатая диаграмма

const barCtx =
document.getElementById('barChart').getContext('2d');

new Chart(barCtx, {
    type: 'bar',
    data: {
        labels: ['Победы белых', 'Победы чёрных', 'Ничьи'],
        datasets: [{
            label: 'Количество партий',
            data: [whiteWins, blackWins, draws],
            backgroundColor: [
                '#f0f0f0',
                '#333333',
                '#888888'
            ],
            borderColor: [
                '#cccccc',
                '#111111',
                '#666666'
            ],
            borderWidth: 1
        }]
    },
    options: {
        responsive: true,
        plugins: {
            legend: {
                display: false
            }
        }
    }
});

```

```

        }
    },
    scales: {
        y: {
            beginAtZero: true,
            ticks: {
                stepSize: 1
            }
        }
    }
}

});
</script>
{% endif %}

</body>
</html>

```

Листинг В.7 — Файл стилей (style.css)

```

* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    background-color: #f5f5f5;
    color: #333;
}

.container {
    max-width: 1100px;
    margin: 40px auto;
    padding: 0 20px;
}

```

```

}

h1 {
    font-size: 28px;
    text-align: center;
    margin-bottom: 8px;
    color: #222;
}

.subtitle {
    text-align: center;
    color: #666;
    margin-bottom: 30px;
    font-size: 15px;
}

/* Секция загрузки */
.upload-section {
    background: #ffffff;
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 24px;
    margin-bottom: 24px;
    text-align: center;
}

.upload-section input[type="file"] {
    margin-bottom: 12px;
    font-size: 14px;
}

.btn-primary {
    background-color: #2c2c2c;
    color: #ffffff;
    border: none;
    padding: 10px 24px;

```

```

        border-radius: 6px;
        cursor: pointer;
        font-size: 14px;
    }

    .btn-primary:hover {
        background-color: #444;
    }

    .btn-secondary {
        background-color: #ffffff;
        color: #666;
        border: 1px solid #ccc;
        padding: 8px 20px;
        border-radius: 6px;
        cursor: pointer;
        font-size: 13px;
    }

    .btn-secondary:hover {
        background-color: #f0f0f0;
    }

    /* Сообщения */
    .message {
        padding: 12px 16px;
        border-radius: 6px;
        margin-bottom: 20px;
        font-size: 14px;
    }

    .message.success {
        background-color: #e6f4ea;
        color: #2d6a4f;
        border: 1px solid #b7dfbf;
    }

```

```

.message.error {
    background-color: #fdecea;
    color: #b00020;
    border: 1px solid #f5c2c7;
}

/* Заголовок статистики */
.stats-header {
    text-align: center;
    margin-bottom: 24px;
}

.stats-header h2 {
    font-size: 20px;
    color: #333;
}

/* Диаграммы */
.charts-section {
    display: flex;
    gap: 24px;
    margin-bottom: 32px;
    flex-wrap: wrap;
}

.chart-box {
    background: #ffffff;
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 20px;
    flex: 1;
    min-width: 280px;
    text-align: center;
}

```

```

.chart-box h3 {
    font-size: 15px;
    margin-bottom: 16px;
    color: #444;
}

/* Таблицы */
.table-section {
    background: #ffffff;
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 20px;
    margin-bottom: 24px;
    overflow-x: auto;
}

.table-section h3 {
    font-size: 16px;
    margin-bottom: 14px;
    color: #333;
}

table {
    width: 100%;
    border-collapse: collapse;
    font-size: 13px;
}

thead tr {
    background-color: #2c2c2c;
    color: #ffffff;
}

thead th {
    padding: 10px 12px;
    text-align: left;
}

```

```
}

tbody tr:nth-child(even) {
    background-color: #f9f9f9;
}

tbody td {
    padding: 8px 12px;
    border-bottom: 1px solid #eee;
}
```

Приложение Г

Скриншоты пользовательского интерфейса.

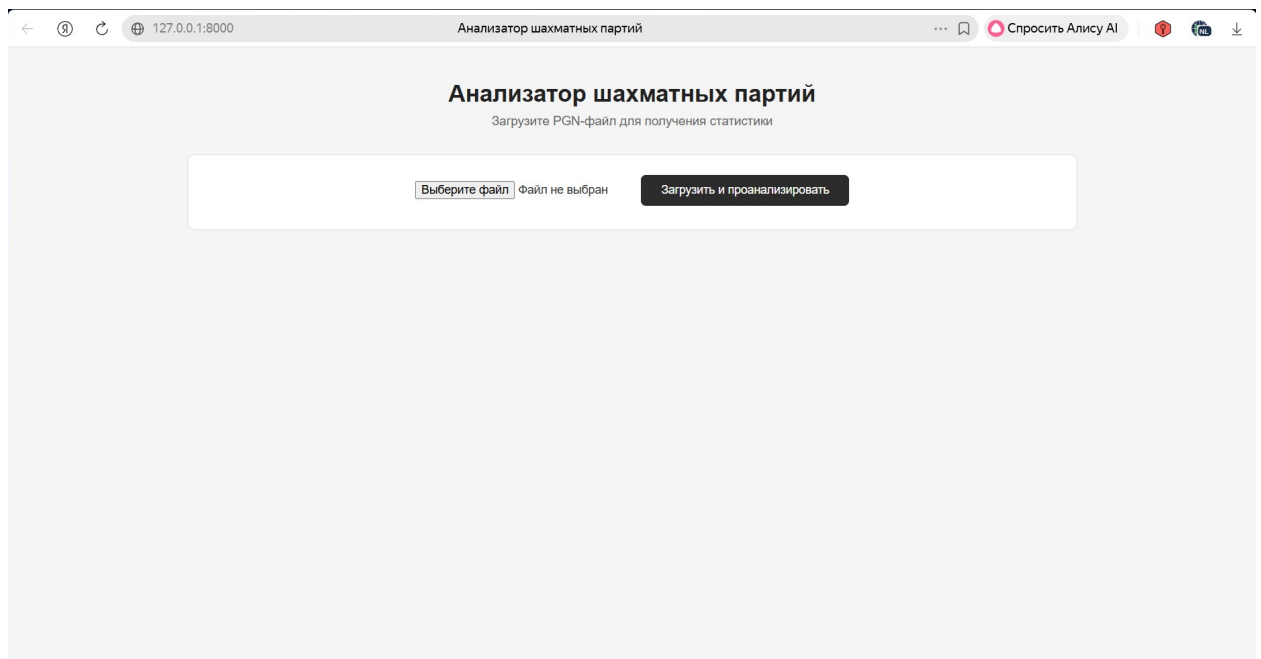


Рисунок Г.1 - Главная страница приложения до загрузки файла

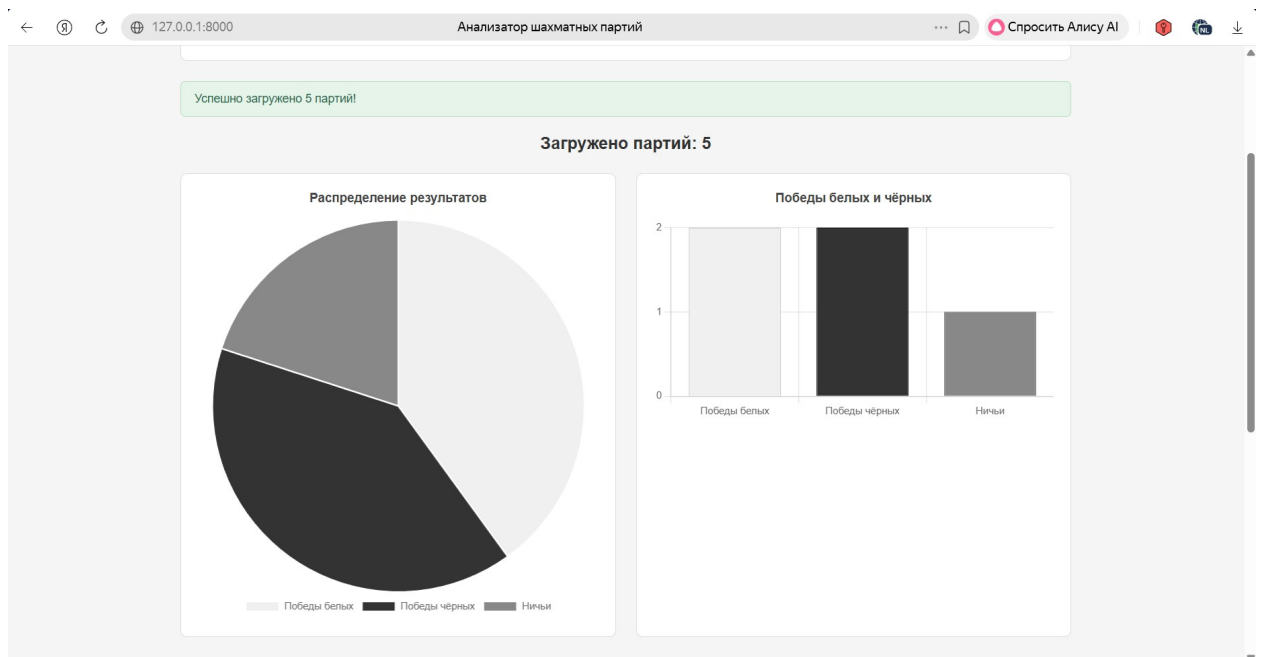


Рисунок Г.2 - Результат успешной загрузки PGN-файла

Топ-5 популярных дебютов

№	Дебют	Количество партий
1	Italian Game	2
2	Reti Opening	1
3	Bishop's Opening	1
4	Barnes Opening	1

Рисунок Г.3 - Таблица наиболее популярных дебютов

Все загруженные партии

№	Белые	Чёрные	Результат	Дата	Дебют	Ходов
1	Ivanov	Petrov	1-0	2026.04.14	Bishop's Opening	7
2	Sidorov	Smirnov	0-1	2026.04.14	Barnes Opening	4
3	Alexeev	Nikolaev	1/2-1/2	2026.04.14	Reti Opening	6
4	Volkov	Morozov	1-0	2026.04.14	Italian Game	15
5	Fedorov	Orlov	0-1	2026.04.14	Italian Game	14

Рисунок Г.4 - Таблица загруженных партий

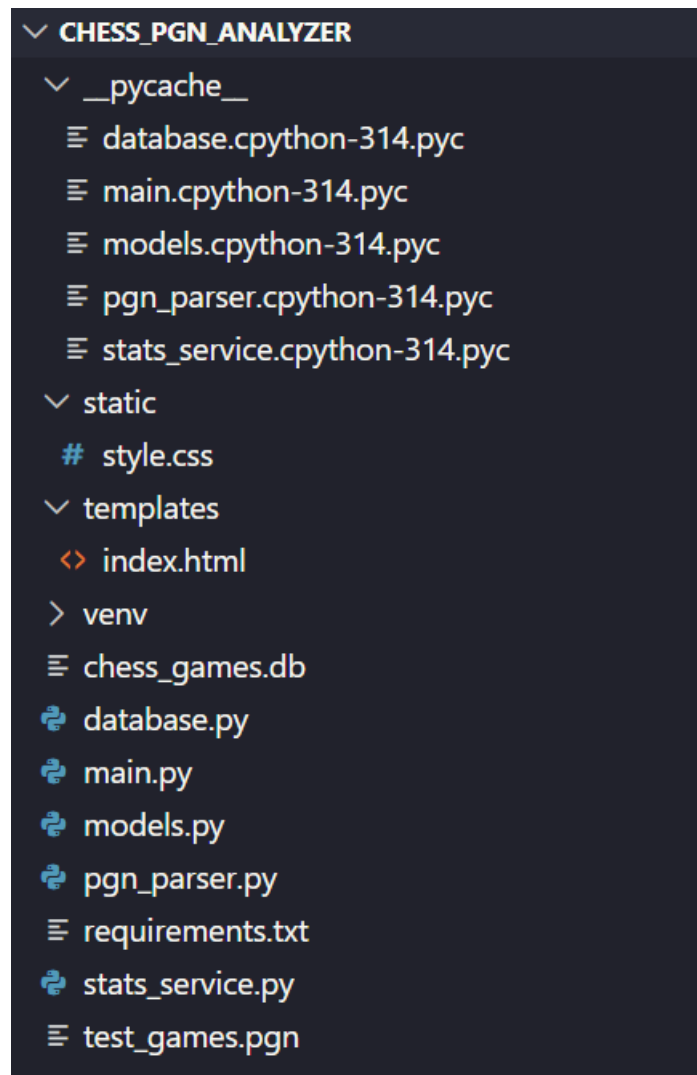


Рисунок Г.5 - Структура файлов проекта в среде разработки VS Code

Приложение Д

Сценарии тестирования.

В рамках проверки работоспособности разработанной информационной системы было проведено функциональное тестирование основных модулей приложения. Тестирование выполнялось вручную с использованием набора подготовленных входных файлов, имитирующих как корректные, так и ошибочные сценарии работы.

Цель тестирования состояла в подтверждении корректности следующих функций:

- загрузка PGN-файлов;
- проверка расширения и содержимого файла;
- парсинг одной и нескольких партий;
- сохранение извлечённых данных в базу;
- формирование статистических показателей;
- отображение диаграмм и таблиц;
- очистка данных перед новой загрузкой.

Для тестирования использовались файлы разных типов: корректные PGN-файлы, пустой файл, файл неверного формата, а также PGN-файлы с неполным набором тегов.

Таблица Д.1

Результаты тестирования основных сценариев

№	Тестовый сценарий	Входной файл	Ожидаемый результат	Фактический результат	Статус
1	Загрузка корректного файла с одной партией	valid_1.pgn	Партия загружается и отображается	Соответствует	Пройден
2	Загрузка корректного файла с 5 партиями	valid_5.pgn	Отображается статистика по всем партиям	Соответствует	Пройден
3	Загрузка корректного файла с 50 партиями	valid_50.pgn	Система обрабатывает все записи без ошибок	Соответствует	Пройден
4	Загрузка пустого файла	empty.pgn	Выводится сообщение об ошибке	Соответствует	Пройден

5	Загрузка файла неверного формата	test.txt	Система отклоняет файл	Соответствует	Пройден
6	Отсутствие тега Opening	no_opening.pgn	Партии сохраняются, поле дебюта пустое	Соответствует	Пройден
7	Отсутствие тега Date	no_date.pgn	Партии сохраняются, поле даты пустое	Соответствует	Пройден
8	Проверка круговой диаграммы	valid_5.pgn	Значения совпадают с рассчитанным и	Соответствует	Пройден
9	Проверка таблицы дебютов	valid_5.pgn	Отображаются наиболее частые дебюты	Соответствует	Пройден
10	Повторная загрузка файла	valid_1.pgn → valid_5.pgn	Старые данные удаляются, новые сохраняются	Соответствует	Пройден

По результатам тестирования было установлено, что система корректно выполняет все основные функции, предусмотренные техническим заданием. При работе с корректными PGN-файлами приложение успешно загружает партии, извлекает данные, сохраняет их в базу и формирует статистику.

Отдельно была проверена устойчивость системы к ошибочным сценариям. При попытке загрузить пустой файл или файл неверного формата приложение не завершает работу аварийно, а выводит понятное сообщение пользователю. Это подтверждает выполнение требований по надёжности.

Кроме того, тестирование показало, что приложение корректно обрабатывает неполные PGN-записи, в которых отсутствуют отдельные теги. Это особенно важно для работы с реальными шахматными архивами, содержащими неоднородные данные.

Ссылка на git проекта?